Complete Specification and Design for Simile

Anthony Hunt

October 17, 2026

Contents

1 High-Level Design **4**
1.1 Overview . . . 4
1.2 Scope . . . 4
1.3 Background . . . 5
1.3.1 Programmers should not worry about implementation details . . . 5
1.3.2 Programmers should focus on correctness, compiler on efficiency . . . 7
1.3.3 Partial computation is powerful . . . 9
1.4 Requirements . . . 9
1.5 Semantics . . . 10
1.6 Experiments . . . 10
1.7 Pipeline Architecture . . . 10
1.7.1 Parser . . . 11
1.7.2 Static Analysis . . . 12
1.7.3 General Rewrites . . . 13
1.7.4 Concretization . . . 14
1.7.5 Specialized Rewrites . . . 15
1.7.6 SSA + LLVM Lowering . . . 15

2 Specification **15**
2.1 Lexicon . . . 16
2.1.1 Whitespace . . . 16
2.1.2 Identifiers . . . 16
2.1.3 Primitives . . . 16
2.1.4 Operators . . . 16
2.1.5 Reserved Keywords . . . 18
2.2 Syntax . . . 19
2.3 Static Analysis - Type System . . . 22
2.3.1 Base Environment Interactions . . . 22
2.3.2 Subtype Interactions . . . 23
2.3.3 The Set Type . . . 23
2.3.4 Primitive Types . . . 24
2.3.5 Collection Types . . . 24
2.3.6 Relation Subtypes . . . 25

2.3.7 Simple Commands . . . 25
2.3.8 Quantification Expressions . . . 26
2.3.9 Boolean Operations . . . 27
2.3.10 Equality and Membership . . . 27
2.3.11 Set Operations . . . 28
2.3.12 Bag Operations . . . 28
2.3.13 Relation Operations . . . 28
2.3.14 Sequence Operations . . . 29
2.3.15 Numerical Operations . . . 29
2.3.16 Record Types . . . 30
2.3.17 Compound Commands . . . 30
2.3.18 Built-in Functions . . . 31
2.4 Static Analysis - Type System Extensions with Traits . . . 31
2.4.1 Available Traits . . . 32
2.4.2 Type-trait interactions . . . 34
2.4.3 Trait-trait Interactions . . . 36
2.4.4 Operation-Trait Interactions . . . 37
2.5 Operational Semantics . . . 54

3 (Abstract) Data Types **72**
3.1 Traits . . . 72
3.2 Boolean . . . 72
3.3 Integer . . . 72
3.4 Float . . . 72
3.5 String . . . 72
3.6 Set . . . 72
3.7 Relation . . . 72
3.8 Sequence . . . 72
3.9 Bag . . . 72
3.10 Tuple . . . 72
3.11 Record . . . 72
3.12 Procedure . . . 72
3.13 Generic . . . 72
3.13.1 Operations . . . 72

4 User-Facing Specification **72**
4.1 (Abstract) Data Types . . . 72
4.1.1 Primitives . . . 72
4.1.2 Sets . . . 73
4.1.3 Relations . . . 74
4.1.4 Bags . . . 75
4.1.5 Sequences . . . 75
4.2 Data Type Operations . . . 76
4.2.1 Primitives . . . 76
4.2.2 Sets . . . 76
4.2.3 Relations . . . 76

4.2.4 Bags . . . 76
4.2.5 Sequences . . . 76

5 AST Lowering and Optimization Rules **76**
5.1 Syntactic Sugar for Bags . . . 76
5.2 Syntactic Sugar for Sequences . . . 76
5.3 Syntactic Sugar for Relations . . . 76
5.4 Comprehension Construction . . . 76
5.5 Disjunctive Normal Form . . . 77
5.6 Or-wrapping . . . 78
5.7 Generator Selection . . . 78
5.8 GSP to Loop . . . 79
5.9 Relational Subtyping Loop Simplification . . . 80
5.10 Loop Code Generation . . . 81
5.11 Replace and Simplify . . . 81

1 High-Level Design

1.1 Overview

This document serves as a living specification and design record of the very-high-level programming language Simile. Simile is founded on the union between formal specification languages and efficiency-focused programming languages, with the belief that a language can be both highly performant and highly abstract. We focus on providing a python-like/specification-language-like programming experience with a compiler strong enough to compete with hand-optimized C code. Where possible, we aim to formalize Simile's compilation process to ensure that aggressively optimized programs remain correct by construction.

Simile takes its syntactic and semantic roots in that of Event-B, a set-theory-first language used to formalize safety-oriented software specifications. Event-B notation often leans on set comprehensions (of the form $\{ x \in S \mid E \}$) and set operations (like $S \cup T$), with progressive specification refinement at the core of the development process. Like object-oriented programs, Event-B programs tend to operate at multiple refinement levels simultaneously, especially when proving program correctness. Each atomic action must be proved correct, then proofs may be composed to prove larger programs, and so on. With Event-B as inspiration, Simile's syntax and semantics is intended to be proof-friendly in the same way, by following discrete mathematics notation.

At the same time, Simile attempts to use the semantic information available at high-level programs to aggressively optimize the executable value. For example, $X \cap T = T$ may require evaluation of $X \cap T$ in a normal programming language, but Simile attempts to match known theorems against expressions to determine that $X \cap T = T$ is only true when $X \supseteq T$. By avoiding the difficult reasoning paths of traditional imperative languages (involving side effects or mutable objects) to perform an action, set theory semantics affords an entire class of optimizations not available to other programming languages.

By building from 50 years of programming language and formal methods literature, we hope to create a language that will spark further interest in simple-to-use, efficient-to-run languages. Following sections include the high-level design of Simile, formalized subsystems, and an implementation reference.

1.2 Scope

In scope:

- We follow set theory semantics most of the time, diverging only for a particularly useful expression or programming objective.
- Programs should remain as abstract as possible, but users may provide helpful information to the compiler to unlock more aggressive optimizations.
- Notation follows that of Event-B (mostly), including a rich set of set and relational operators.

Out of scope:

- We cannot hope to become the objectively best language in all use cases. Each language has their strength, and Simile aims to be well-rounded; ergonomic enough to pick up, and fast enough to be more usable than other specification language runtimes.
- We do not follow conventional imperative language semantics that would otherwise clash with set theory optimizations.

1.3 Background

1.3.1 Programmers should not worry about implementation details

In high level programming languages, users often reach for abstract constructs like relations (dictionaries in Python, maps in Java), lists, and sets, largely because they are standardized, concise, and expressive. For example, HashMaps can be found in almost every language's standard library, and it would be peculiar to find a language that does not have first-class support for list collections. Most of the time, programmers using these types don't think about the layout or execution of their data, just that it is held within a container and operable under several composable actions (concatenation, union, intersection, lookups, etc.). However, this abstraction under the semantics of traditional high-level imperative/OO programs can lead to wasted memory and computation time, especially when composing multiple operations together. For example, the expression $S \cap (A \cup B \cup C)$ is theoretically bounded by the size of S, but a conventional language would construct several intermediary sets for $A \cup B$, then $(A \cup B) \cup C$, consuming more memory and time than required.

Low-level languages share an adjacent problem: programmers may trust primitive data type operations to be fast, but they must also tune every piece of their code to fit the type's implementation. Formally simple and powerful operations like list concatenation force the user to consider implementation details like memory allocation and ownership for both inputs and outputs. For example, in $A := S +\!\!+ T$, does the specific implementation create a new list for A containing a shallow copy of the contents of S and T? Or perhaps A and S will reference the same object, where the data in T is moved to the pointed-to value of S. Programmers must track every byte of data for straightforward operations, causing mental overhead, distractions from program correctness, and the risk of implementation errors.

Abstract data types are semantically powerful and have been implemented by hundreds of languages across thousands of libraries. Further, they can be represented by a variety of concrete types, with wildly varying levels of efficiency and effectiveness for different use cases. For example, a set of integers can be implemented by a HashSet, a bitset, an array, a tree set, or even a trie. HashSets are advantageous for element lookups, but may not be able to compare to a dense bitset in terms of memory efficiency. An array could be effective for a set that may need to be sorted, but doesn't have direct element lookups. With a little metadata-help from the programmer, the compiler should be intelligent enough to make decisions on the concrete type and optimize expressions tailored to the concrete type while completely following the abstract type's semantics.

Motivating Example: A Visitor Information System Let's turn towards an example for specification-focused programming: a group of conference administrators create a program to track resourcing for a week's worth of workshops. They already have a list of visitors, available rooms, and planned workshops for each room. A specification would model these records via carrier sets (i.e., enumerated sets):

$$Visitor = \{\, Ryan, Adrian, Kevin, Charlie, Michael, ... \,\}$$
$$Room = \{\, 100, 101, 201, ... \,\}$$
$$Workshop = \{\, ESOP, FASE, FoSSaCS \,\}$$

To make this model more useful, we express relationships between these sets. For example, every workshop must occur in a room and visitors will attend workshops. Further, we can place restrictions on each relation concisely through specification language notation. If we know workshops occur in a room, it would be safe to tighten our observation that *all* workshops must occur in one room *at a time*. So, we can explicitly refine the relationship to an injective (one-to-one) function that is total on the domain (the carrier set $Workshop$). In formal notation, we write concisely:

$$location : Workshop \rightarrowtail Room$$

We can further refine another relationship: visitors can attend workshops, but visitors can only attend one workshop *at a time* and *not all* visitors may attend a workshop. This relation is actually partial on the carrier set of $Visitor$ and a

functional (many-to-one) relation. Formally:

$$attends : Visitor \nrightarrow Workshop$$

With these relations, we can query for interesting information concisely. Let's say we want to throw a surprise dinner for all attendees in a specific room. We can compose the *attends* relation with the *location* relation to find the number of people expected in a specific room. For a specific *room*, we write:

$$card((location^{-1} \circ attends^{-1})[\{\, room \,\}])$$

A HashMap would be the first concrete data structure of choice for most programmers, but without knowledge of the refined relationships, we cannot guarantee that the relational inverse operation is guaranteed to work. We set aside HashMaps for now for the sole reason that they are limited in their representation capabilities; we cannot model a many-to-many relationship using the conventional definition of the type.

If we were to build a compiler for this specification language, a naive implementation might only see the broad connection between these sets and choose to safely model them as a HashSet of tuples (which can represent many-to-many relations). In this case, we concretize the types of *location* and *attends* like so:

$$location : HashSet[(Workshop, Room)]$$
$$attends : HashSet[(Visitor, Workshop)]$$

Then, each operation from the above query can be ordered in an imperative fashion:

$locationInverse : HashSet[(Room, Workshop)] := Inverse(location)$
$attendsInverse : HashSet[(Workshop, Visitor)] := Inverse(attends)$
$visitorsInRooms : HashSet[(Room, Visitor)] := Compose(locationInverse, attendsInverse)$
$visitorsInQueryRoom : HashSet[Visitor] := Lookup(room, visitorsInRooms)$
$mealsToPrepForRoom : int := Card(visitorsInQueryRoom)$

With a library of concrete functions:

function INVERSE($r : HashSet[(X, Y)]$) ▷ Runtime: $O(n)$. Memory use: $O(n)$
  $r' : HashSet[(Y, X)] := \{\ \}$
  for $x, y \in r$ **do**
    $r'.add((y, x))$ ▷ $s.add$ is a primitive HashSet operation mutating s
  return r'
function COMPOSE($r_1 : HashSet[(X, Y)], r_2 : HashSet[(Y, Z)]$) ▷ Runtime: $O(n^2)$. Memory use: $O(n)$
  $r' : HashSet[(X, Z)] := \{\ \}$
  for $x, y_1 \in r_1$ **do**
    for $y_2, z \in r_2$ **do**
      if $y_1 = y_2$ **then**
        $r'.add((x, y))$
  return r'
function LOOKUP($searchFor : X, r : HashSet[(X, Y)]$) ▷ Runtime: $O(n)$. Memory use: $O(n)$
  $r' : HashSet[Y] := \{\ \}$
  for $x, y \in r$ **do**
    if $searchFor = x$ **then**
      $r'.add(y)$
  return r'
function CARD($s : HashSet$) ▷ Runtime: $O(n)$. Memory use: $O(1)$

$c : int := 0$
for $x \in s$ **do**
$c := c + 1$
return c

As evident by the poor runtime and memory complexity, this is far from the most efficient representation of the system. However, it is a direct, valid translation, fulfilling all semantic requirements. A human may never write such code, but a compiler without statically analyzing the use of abstract times, may choose the broadest representation at the cost of runtime and memory usage. With knowledge of the problem domain and usage of these library functions, human programmers can intuitively decide that the *Lookup* function, for example, only ever needs to match for one room. Further, they would likely observe that this solution could be made a magnitude faster by using HashMaps instead of a tupled HashSet.

But as programmers deciding on concrete data structures, it is our responsibility to be aware of the restrictions of the concrete types and make the most appropriate decision for the expected data. A compiler should be able to make the same decisions, consulting a longer list of alternatives for further unique cases. If we inform the compiler that HashMaps must represent many-to-one relations in addition to the refined relational types mentioned above (where *location* and *attends* are one-to-one and many-to-one respectively), we effectively offload the burden of implementation choice towards a rigid, correct system.

Compare the above solution with the optimal HashMap implementation, which allocates no extra space for intermediate sets and runs in $O(n)$:

function HashMapSolution($location : HashMap[Workshop, Room], attends : HashMap[Visitor, Workshop]$)
c := 0
for $v, w \in attends$ **do**
if $location[w] = room$ **then**
$c := c + 1$
return c

Digression: relational databases Upon reviewing this example, we see strong parallels in language and functionality to relational database engines (particularly SQL). And as a data processing-focused problem, this example could of course be modelled by a relational database. For a very large event with many rooms, visitors, and workshops, perhaps a relational database is the only way to manage the data. However, using formal notation, the programmer is not burdened with the choice of that representation; they focus on correctness of the abstract data and abstract operations. The specification language intentionally hides implementation-specific operations like joins and counts, instead opting for a purer mathematical model following traditional set theory semantics.

The execution engine should have the freedom of choosing a relational database-like storage-and-query mechanism, but it should also have the freedom to choose between aggressively memory-efficient or fully-cached implementations too. In this particular example, a relational database may not even be the best choice of representation. Imagine we could encode the unchanging carrier sets (ex. a building has a limited number of *Room*s, the set of $Workshop$s is naturally limited by the number of *Room*s) as a compact bitset. If we were to perform an intersection between these encoded sets, the execution could be many times faster than the overhead from a relational database.

By providing the compute engine with more opportunities to follow unique optimization paths, we may be able to tune the computation method to the problem at hand.

1.3.2 Programmers should focus on correctness, compiler on efficiency

One of the best traits of specification languages towards correct program development is their emphasis on proving the intent of the code. Even in industry, organizing software requirements remains a bottleneck for shipping features and products. Although software developers will no doubt need to consider algorithmic efficiency when writing programs, they

should be able to abstract away primitive operations to focus on writing readable, correct, and clear code. Then, the compiler should have the freedom to swap out expressive code with a semantically equivalent, more efficient counterpart. In this case, rigorous semantics are required to demonstrate that the source program exhibits the same key behaviours as optimized programs.

High level program libraries and frameworks are already trending towards semantically rich, abstract interfaces covering a low level hand-optimized implementation. For example, the numpy library has become essential syntactic sugar for scientific computing within python. Python further serves as the standard for machine learning tooling, with libraries like PyTorch, TensorFlow, and Triton abstracting complex GPU code behind a vendor-agnostic interface.

By simplifying the language and formalizing semantics, we can grant the compiler more power over optimizations than would otherwise be possible in conventional languages.

Motivating Example: Operation Optimization in Python Let's say we have 3 sets S, T, V that can only be represented by the concrete HashSet type. Further, take our candidate query $(S \cup T) \cap V$. Python provides a suite of abstract operators for sets, so this expression may be represented as `(S | T) & V`. But its imperative semantics hide the actual computation process:

```
def compute(S, T, V):
    # Step 1: S | T
    s_union_t = set()
    for x in S:
        s_union_t.add(x)
    for x in T:
        s_union_t.add(x)
    # len(s_union_t) <= len(S) + len(T)

    # Step 2: (S | T) & V
    ret = set()
    for x in s_union_t:
        ret.add(x)
    for x in copy(ret):
        if x not in V:
            ret.remove(x)
    # len(ret) <= len(V)
    return ret
```

Because the expected behaviour of Python operators is to create a new set while leaving the input arguments untouched, the interpreter must create a new set for the union of S and T, then another set for the intersection with V. Further, Python is forced to evaluate from left-to-right, obeying the priority of parenthesis. So, in the likely case where $S \cup T$ are larger than V, we must waste computation and memory adding and removing values from the return set (as in Step 2). The semantics of conventional imperative languages limit the compiler's ability to shuffle expression evaluation, not least because of side effects.

If we freed the semantics of these operators so that known set theorems can replace expressions, we may choose to swap $(S \cup T) \cap V$ with a more complex but computationally faster expression $(S \cap V) \cup ((T \setminus S) \cap V)$. Further, if we prevent the construction of intermediary sets and instead compute the final result directly, we can create an implementation where the resulting set never undoes prior operations, and never consumes more memory than necessary:

```
def compute(S, T, V):
```

```
    # Step 1: S & V
    ret = set()
    for x in S:
        if x in V:
            ret.add(x)
    # len(ret) <= min(len(S), len(V))
    # Step 2
    for x in T:
        if x not in S and x in V:
            ret.add(x)
    # len(ret) <= min(len(S) + len(T), len(V))
    return ret
```

The constructed return set strengthens the upper bound cardinality by distributing the intersection operation (which shrinks set sizes) across the union operation (which grows set sizes). As a bonus for focusing on memory efficiency, the runtime is faster too. This second version only iterates over S and T once, while the first version iterates over S, T, and $S \cup T$ twice.

High level specifications can make the intention of computation clearer and simpler, allowing simplifications to happen early in the compilation process. But we must be careful: as demonstrated with this example, the optimized answer is not necessarily the shortest formula. We must consider the concrete type representations, the atomic actions of those concrete types, and the strengths/weaknesses of those atomic actions. A new language defining only abstract semantics allows for extensions of optimizations without any behavioural differences the user would care about; the only behaviours we promise are those defined by the semantics.

1.3.3 Partial computation is powerful

Motivating Example: Maximum Tail Sum

1.4 Requirements

The below requirements outline the driving features of Simile:

R1 Syntax is that of a very high level language, inspired by formal languages but practical for imperative programming.

R2 Set theory semantics remain consistent through any optimization. The semantic contract is defined by formalized behaviours, and the compiler must uphold these contracts.

R3 Programmers think in abstract types; the compiler has the responsibility of concretization. Type system behaviours are formalized and explicit.

R4 Programmers must be able to communicate with the compiler when needed (to direct optimizations), but they should never be forced to do so.

R5 Optimizations are described through a term rewriting system, though we do not necessarily prove these formalizations. We lean on existing optimizers when appropriate (ex. LLVM as the last stage of compilation)

R6 Output should be efficient in memory and runtime, competitive with optimized high-level languages like Java or CPython. The overall goal is to be competitive with low level languages like C and Rust.

1.5 Semantics

A basic strategy to optimize set and relational expressions is:

1. Normalize the expression as a set comprehensions
2. Simplify and reorganize conjuncts of the set comprehension body

The TRS for this language primarily involves lowering collection data type expressions into pointwise boolean quantifications. Breaking down each operation into set builder notation enables a few key actions:

- Quantifications over sets ($\{\, G \cdot P \mid E \,\}$) are naturally separated into generators (G), (non-generating) predicates (P), and expressions (E). For sets, at least one membership operator per top-level conjunction in G will serve as a concrete element generator in generated code. Then, top level disjunctions will select one membership operation to act as a generator, relegating all others to the predicate level. For example, if the rewrite system observes an intersection of the form $\{\, x \in S \wedge x \in T \mid x \,\}$, the set construction operation must iterate over at least one of S and T. Then, the other will act as a condition to check every iteration (becoming $\{\, x \in S \cdot x \in T \mid x \,\}$).
- By definition of generators in quantification notation, operations in G must be statements of the form $x \in S$, where x is used in the "element" portion of the set construction. Statements like $x \notin T$ or checking a property $p(x)$ must act like conditions since they do not produce any iterable elements.
- Any boolean expression for conditions may be rewritten as a combination of $\neg, \vee$, and $\wedge$ expressions. Therefore, by converting all set notation down into boolean notation and then generating code based on set constructor booleans, we can accommodate any form of predicate function.

1.6 Experiments

1.7 Pipeline Architecture

Like most compiled programming languages, Simile follows the usual workflow of parsing, analysis, optimization, and lowering. While the pipeline itself is not completely novel, the combination of Simile's semantics with verifiable components allow for aggressive, safe optimizations. Figure 1 shows the end-to-end flow of code through the compilation pipeline.

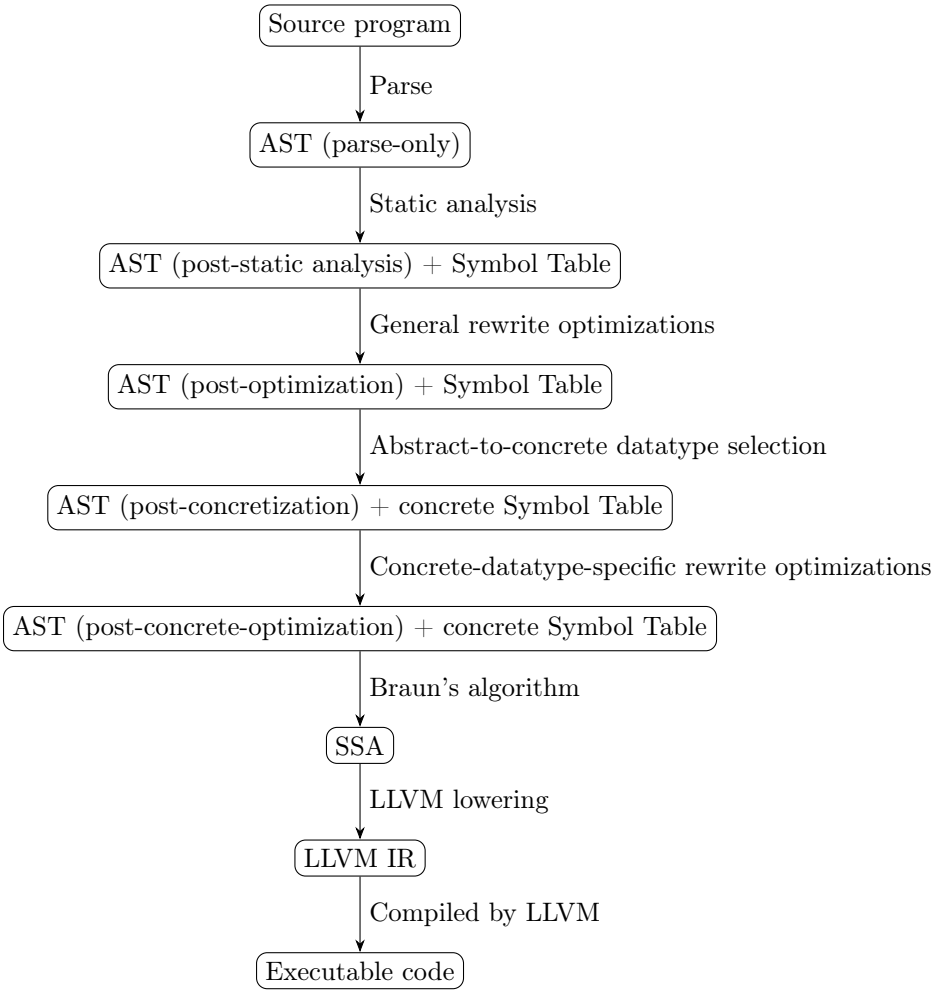


Figure 1: Simile compilation pipeline from source code to LLVM-based executable

Below sections will discuss each component.

1.7.1 Parser

Simile's lexer uses a maximal munch approach to tokenization for extensibility and straightforward handling of a large symbolic language. However, this comes with the caveat that reuse of underlying characters may be limited. For example, consider the symbol for an injective relation: `>->`, which combines the conventional "greater-than" and "negative" symbols from integer arithmetic. Since our scanner does not implement backtracking, the expression `2>-1` may see the combined `>-` as an incomplete formation of `>->`, rather than the expected $2 > -1$. To fix this, we require spaces before and after overloaded characters (until backtracking is implemented). The full list of symbols is contained in Section 2.1.

Parsing follows a conventional recursive descent style, hand-rolled for better error messages compared to parser generators. The full grammar is in Section 2.2 - note that left-recursive productions are parsed as right-recursive, then reformed

into the expected left-recursive structures. Pretty printers are available and re-parsable for the AST throughout the optimization process that convert ASCII symbols into their unicode counterparts.

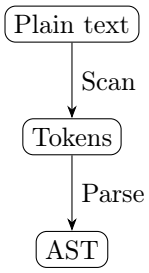


Figure 2: Component Diagram: Parser

1.7.2 Static Analysis

Static analysis is the backbone of type and trait resolution.

First, the AST is normalized from generic "BinaryOp" and "UnaryOp" classes with operation fields into proper operation classes; this has the dual effect of simplifying match statements from generic `BinaryOp(...)` cases to a specific operation and guaranteeing that we explicitly define an AST node for each operator, especially considering Python's dynamic type system. After each AST-modifying pass, we rerun normalization to strengthen the above pipeline guarantees.

Simile's standard library is then prepended to the input AST as a wildcard import, containing function definitions referenced throughout this document (ex. `min`, `max`, `min_map`).

The symbol table contains two supporting mappings: a scope map to capture the environment a symbol is located in (with references to the parent scope and available symbols), and a large symbol map assigning an identifier, scope, and type to each declared name. Scoping rules are captured formally by typing and operational semantic judgements in Section 2, but in plain terms, procedures, quantifications (and comprehensions), conditional statements, lambdas, imports, and records create a static scope based on the environment at declaration time. The symbol table is populated when a fresh variable is declared, either by a quantifier, procedure/record/lambda definition, or assignment. Once the symbol table has been populated, the AST should no longer contain plain-string references to identifiers, rather they should contain pointers to entries in the symbol table.

The type resolver is made up of 3 components: the type annotation resolver, the trait resolver, and the expression resolver. Annotation resolution happens simultaneously with symbol table population, since declared types may be used after declaration. Type annotations may consist of built-in types, record types, tuples of types, and instantiated generic types. Traits are similar, with most appearing as flag options, but specific traits can also be disabled or concretized to effectively control optimization paths. With this information, the expression resolver can derive the type of any expression on-the-fly.

The symbol table, expression resolver, and modified AST are bundled together as an output.

One caveat: The type resolution algorithm follows a direct bottom-up approach as opposed to a more thorough Hindley-Milner system. All declared Simile variables must have an explicit type at definition time, we do not provide implicit types. Future work will relax this hard constraint.

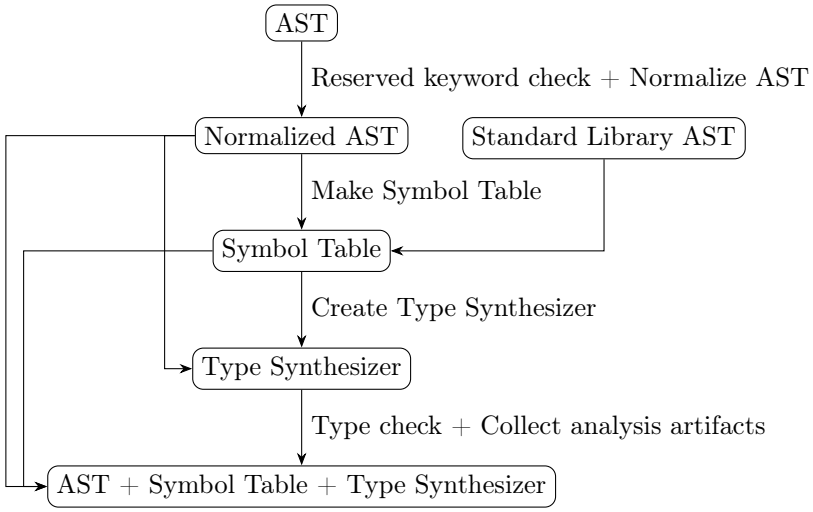


Figure 3: Component Diagram: Static Analysis

1.7.3 General Rewrites

Following in the footsteps of formal languages and theorem provers, Simile defines transformations of terms through a syntactic rewrite system. In its most basic form, rewrites are mappings from one term structure (AST node) to another, applied in a non-deterministic order. For example, a valid rewrite may look like: $x + x \rightsquigarrow 2 \times x$. Simile uses an altered form, where rewrite rules are grouped into collections and may have additional conditions before matching. While simple, rewrite systems are turing complete and thus system developers are concerned with termination and consistent outputs (confluence). However, it retains the ability to formalize and prove that rules are behaviour preserving, ensuring compiler correctness.

Because Simile is focused on abstract data types as the primary compute vehicle, the rewrite optimizer has been split: one section for abstract rewrites, and another for concrete data type rewrites. For example, a relation may be represented as a hashset of tuples or a bi-directional hashmap, but these data structures have differing atomic actions and algorithmic advantages. Thus, the first collection of rewrites is more for high-level concerns and can be written in Simile itself. Standardization and partial algorithmic computation are the focus, where rewrites are only concerned with the abstract definitions of types rather than concrete demands.

To simplify the process of creating a rewrite library, we developed a small DSL (called SimRW) to structure rewrite rules. We declare variables, the rewrite rule itself, and any additional guards required to pass before we can execute a rewrite. For example, simplifying a constant sum can be written as:

```
rule constant_sum:
  vars:
    c: int
    s: set[int]
  rewrite:
    sum x in s | c
    ~>
    card(S) * c
  when:
```

```
does_not_occur_in(x, c)
```

Rules are applied in groups, with complete type and trait information available in guard conditions.

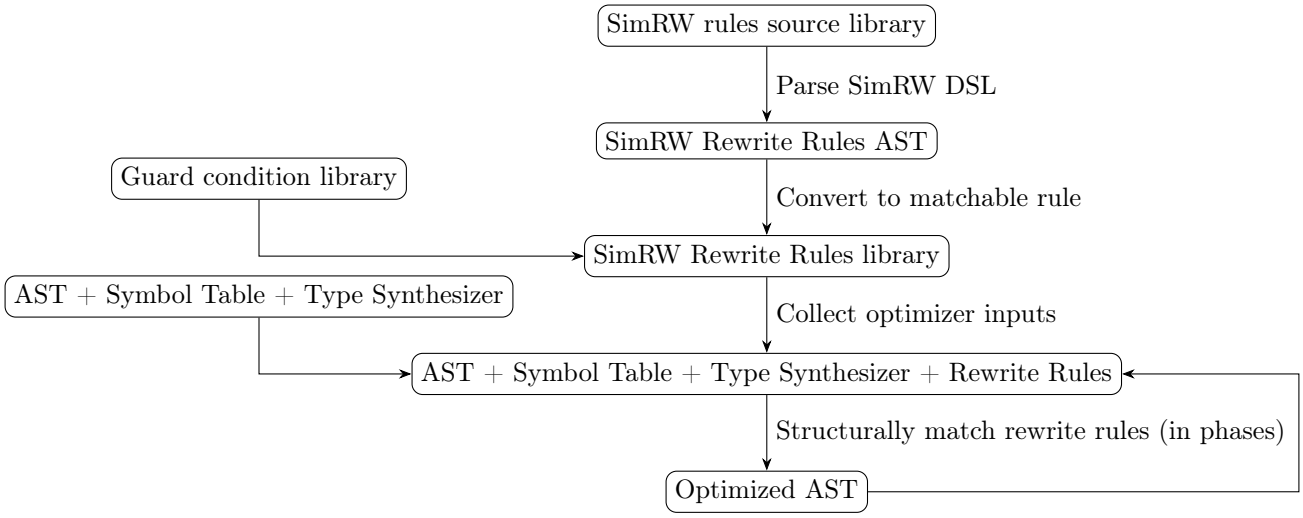


Figure 4: Component Diagram: General Rewrite Optimization

1.7.4 Concretization

Once preliminary optimizations are complete, we must lower the AST into something closer to executable code. We perform more semantic passes over the symbol table, with rudimentary operator counts and trait analysis. Then, from an extensible collection of abstract-to-concrete data type mappings, we use all available semantic information to direct the concrete representation. For example, an abstract set of integers with a tight range and intersection-only operations may be best represented as a bitset.

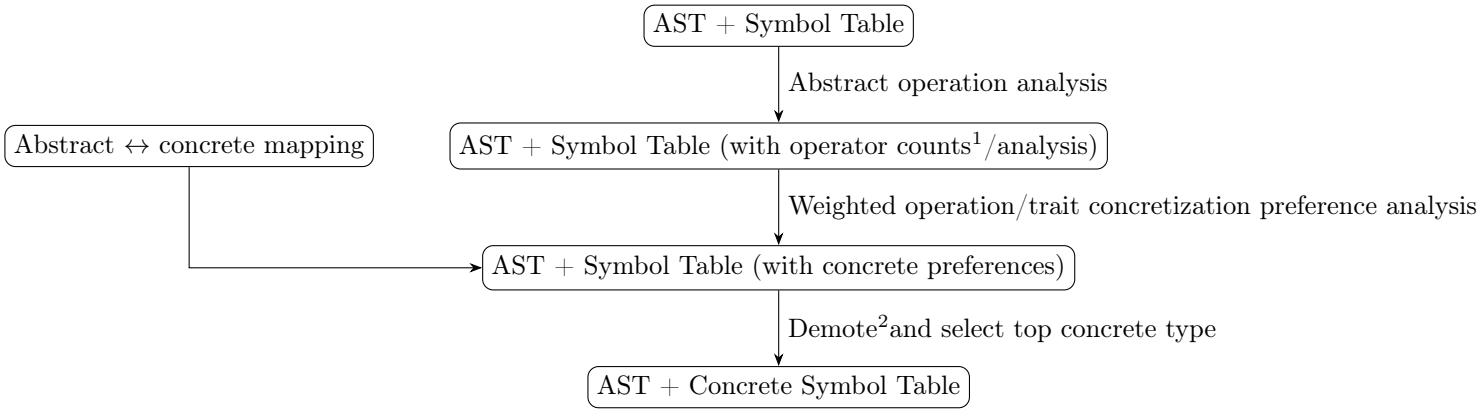


Figure 5: Component Diagram: Abstract Data Type Concretization

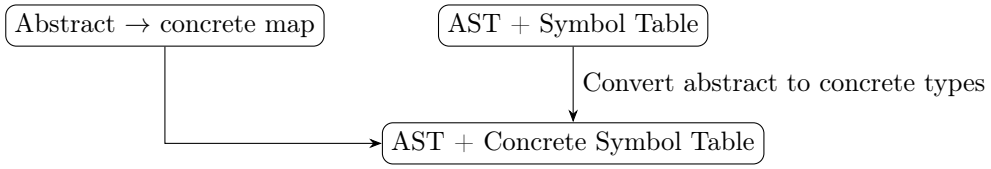


Figure 6: Component Diagram: Abstract Data Type Concretization (Scrappy version)

1.7.5 Specialized Rewrites

Specialized rewrites are similar to the generic counterpart, but combines operation lowering with concrete-specific optimizations. Generally, concrete data types have a set of "atomic" operations used to build towards the abstract types programmers use. For example, the union of two hashsets may involve looping over one hashset and inserting the other's data one-by-one, whereas the intersection of bitsets require one bitwise-AND operation.

Where possible, rules will be written in SimRW, but since concrete types often have unique mutating operations, most of these rewrites will be encoded into Python match statements.

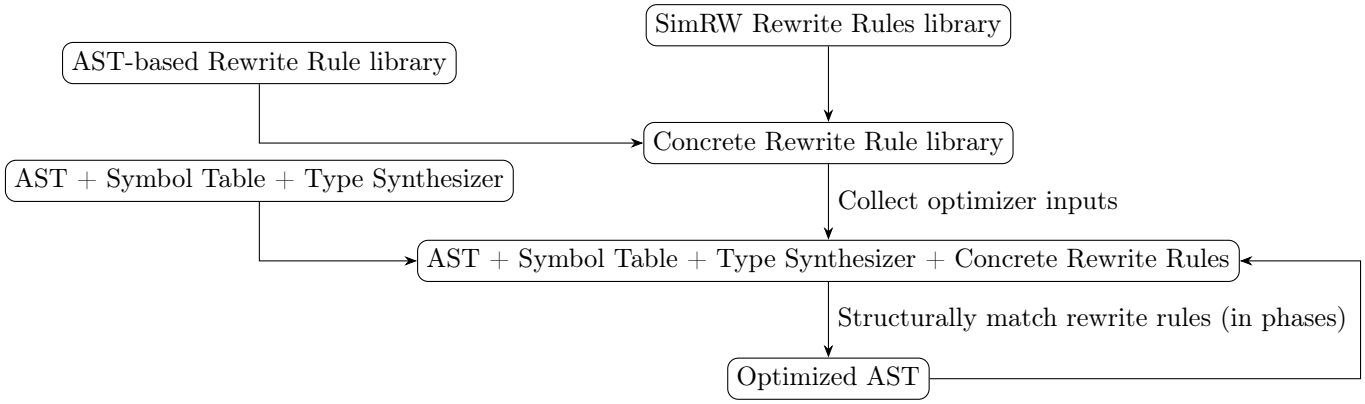


Figure 7: Component Diagram: Concrete-specific Rewrite Optimizations

1.7.6 SSA + LLVM Lowering

After the optimization dust settles, we perform the necessary transformations to convert concretized Simile directly into LLVM output, with no additional optimization efforts.

2 Specification

This section contains the complete specification and low-level design of Simile, inspired by Event-B and Python.

[1] For example, the number of times a variable is used in a particular operation. We traverse the entire tree until that variable is out of scope (inefficient, but should give us somewhat-accurate counts). The most important info is probably which counts are zero, since we know we won't need that operation when concretizing.

[2] A variable's preferences may be demoted based on other operand preferences, following the most conservative type.

2.1 Lexicon

Since Simile makes extensive use of Event-B-like set notation and otherwise non-ASCII characters, this section contains a list of such characters, translations to ASCII symbols where suitable, and other useful lexing information.

2.1.1 Whitespace

Similar to Python, indentation and newline tokens are significant in Simile. In particular, nested code blocks, as in if-statements, for-loops, and procedure definitions, require the use of indentations to signify the beginning and end. Indentation/newlines may also be used within collection-type comprehensions or enumerations, similar to the style seen in JSON. Comments are single-lined and start with a pound symbol (#). Indentation is ignored within bracketed expressions[3].

2.1.2 Identifiers

Identifiers in Simile follow the standard regular expression, using only ASCII letters, underscores, and numbers: $[a-zA-Z_][a-zA-Z_0-9]*$.

2.1.3 Primitives

Integers and floating point numbers follow a similar style to modern programming languages. The dash character ($-$) may be used in either subtraction or as a unary negation. Superscript numbers may be used for constant exponentiation. Booleans (true/false) follow Python style, with capitalized first letters. Strings are delimited with double-quotes (") and may be multilined.

2.1.4 Operators

ASCII tokens will be provided as an alternative to unicode format where suitable. The following are symbols used for expressions:

Token	ASCII Token	Name
$\cdot$	`.:`	Center Dot (Quantifier Separator) [4]
.		Dot
,	`iter_and`	Comma
‘	`iter_or`	Comma
:		Colon
;		Semicolon
|		Vertical Bar
λ	`lambda`	Lambda
(		Left Parenthesis
)		Right Parenthesis
$\langle$	`<<`	Left Bracket
	`[`	Left Bracket (Alternative)
$\rangle$	`>>`	Right Bracket

[3] Except for within the ‘iter’ block. This is a special case handled post-lexing: ignore indentation when parenthesis/bracket/brace level > 1 and we are not within the block portion of ‘iter’... VBAR NEWLINE INDENT ‘block’ DEDENT.

[4] Anywhere $\cdot$ is valid, . is valid, except the parser may attempt to treat . as a record field access instead of a quantifier separator (like $a.b$), so we prefer the use of $\cdot$ instead. NOTE: We plan to remove this in favour of . :

Token	ASCII Token	Name
	`]`	Right Bracket (Alternative)
$\{$		Left Brace
$\}$		Right Brace
$[\![$	`<[`	Left Double Bracket
$]\!]$	`]>`	Right Double Bracket
$=$		Equals
$\neq$	`!=`	Not Equals
$\neg$	`not`	Not
	`!`	Not (Alternative)
$\wedge$	`and`	And
$\vee$	`or`	Or
$\Rightarrow$	`==>`	Implies
$\equiv$	`==`	Equivalent
$\not\equiv$	`!==`	Not Equivalent
	`+`	Plus
	`-`	Minus
	`*`	Multiplication
	`/`	Division
	`div`	Integer Division
	`mod`	Modulo
	`^`	Exponent
$<$		Less Than
$\leq$	`<=`	Less Than or Equal
$>$		Greater Than
$\geq$	`>=`	Greater Than or Equal
$\in$	`in`	In (Membership)
$\notin$	`not in`	Not In (Membership)
$\cup$	`\/`	Union
$\cap$	`/\`	Intersection
$\setminus$	`\`	Backslash (Set Minus)
$\times$	`><`	Cartesian Product
$\subset$	`<<:`	Subset
$\subseteq$	`<:`	Subset or Equal
$\supset$	`:>>`	Superset
$\supseteq$	`:>`	Superset or Equal
$\not\subset$	`!<<:`	Not Subset
$\not\subseteq$	`!<:`	Not Subset or Equal
$\not\supset$	`!:>>`	Not Superset
$\not\supseteq$	`!:>`	Not Superset or Equal
$\mapsto$	`|->`	Maplet
$\oplus$	`<+>`	Relation Overriding
$\circ$	`<>`	Composition
$+\!\!+$	`++`	Concatenation
$^{-1}$	`~`	Inverse
$\lhd$	`<|`	Domain Restriction

Token	ASCII Token	Name
⩤	`<<|`	Domain Subtraction
▷	`|>`	Range Restriction
⩥	`|>>`	Range Subtraction
↔	`<->`	Relation
⇿	`<<->`	Total Relation [5]
↔	`<->>`	Surjective Relation [6]
⇿	`<<->>`	Total Surjective Relation [7]
⇸	`+->`	Partial Function
→	`-->`	Total Function
⤔	`>+>`	Partial Injection
↣	`>->`	Total Injection
⤀	`+->>`	Partial Surjection
↠	`-->>`	Total Surjection
⤖	`>->>`	Bijection
..		Upto
⋃	`union`	General Union
⋂	`intersection`	General Intersection
∑	`sum`	Summation
∏	`product`	Product
∀	`forall`	For All
∃	`exists`	There Exists
↑	`max`	Max
↓	`min`	Min
	`fold`	Fold
	`iter`	Iter

The following are symbols used for programming constructs:

(Unicode) Token	(ASCII) Token	Name
:=	`:=`	Assignment
:∈	`::`	Choice (Non-deterministic Membership) Assignment
→	`->`	Right Arrow

2.1.5 Reserved Keywords

The following list of symbols and identifiers are reserved for programming constructs (in addition to the above expression symbols):

[5] Brave Sans Mono unicode: U+E100
[6] Brave Sans Mono unicode: U+E101
[7] Brave Sans Mono unicode: U+E102

true	false	if	else	for
while	record	enum	procedure	is
is not	return	break	continue	from
import	skip	trait	type	

And for built-in types:

int	float	string	bool	$\mathbb{Z}$	$\mathbb{N}$
$\mathbb{N}_1$	set	sequence	bag	relation	generic
tuple					

And built-in procedures:

map_min	map_max	choice	dom	
ran	card	size	cast	cast_with
print	map	heads	tails	

2.2 Syntax

We follow the standard EBNF notation for describing the language structure.

Start ::= Statements

Statements ::= {SimpleStatements | CompoundStmt}

SimpleStatements ::= SimpleStmt {‘`;`’ SimpleStmt} NEWLINE

SimpleStmt ::= AssignmentOrExpr | ControlFlowStmt | ImportStmt

AssignmentOrExpr ::= Expr [‘`:`’ TypeExpr] [(‘:=’ | ‘:∈’) Expr [TraitStmt]]

TraitStmt ::= ‘`trait`’ Expr {‘`trait`’ Expr}
| NEWLINE INDENT ‘`trait`’ Expr NEWLINE {‘`trait`’ Expr NEWLINE} DEDENT

Expr ::= Quantification | Predicate | LambdaDef

LambdaDef ::= ‘λ’ [FlatTupleIdentifier] ‘·’ Predicate ‘`|`’ Expr

Quantification ::= (‘∑’ | ‘∏’ | ‘⋃’ | ‘⋂’ | ‘∃’ | ‘∀’ | ‘↑’ | ‘↓’) (AtomFollow | QuantificationBody)
| ‘`fold`’ (AtomFollow | Assignment ‘`:`’ QuantificationBody)
| ‘`iter`’ (AtomFollow | IterQuantificationBody)

QuantificationBody ::= BranchQuantificationBody
| Generator {‘`,`’ Generator} (‘`,`’ BranchQuantificationBody | ‘`|`’ Expr)

BranchQuantificationBody ::= ‘`(`’ QuantificationBody ‘`)`’ {‘`(`’ QuantificationBody ‘`)`’}

Generator ::= IdentList ‘∈’ Expr [‘·’ Expr]

IterQuantificationBody ::= BranchIterQuantificationBody
  | GeneratorWithAssignments {‘,’ GeneratorWithAssignments } (‘,’ BranchIterQuantificationBody | ‘->’ IdentList ‘|’ IterBlock)

BranchIterQuantificationBody ::= ‘(’ IterQuantificationBody ‘)’ {‘(’ IterQuantificationBody ‘)’ }

GeneratorWithAssignments ::= Generator ‘:’ Assignment { ‘;’ Assignment}

Assignment ::= Expr [‘:’ TypeExpr] (‘:=’ | ‘:∈’) Expr

IterBlock ::= SimpleStmt {‘;’ SimpleStmt}
  | NEWLINE INDENT Statements DEDENT

IdentList ::= IdentListItem {‘,’ IdentListItem}

IdentListItem ::= IDENTIFIER
  | IdentListItem ‘↦’ IdentListItem
  | ‘(’ IdentList ‘)’

Predicate ::= Implication
  | Predicate (‘≡’ | ‘≢’) Implication

Implication ::= Disjunction
  | Disjunction {‘⇒’ Disjunction}

Disjunction ::= Conjunction {‘∨’ Conjunction}

Conjunction ::= Negation {‘∧’ Negation}

Negation ::= AtomBool
  | ‘¬’ Negation

AtomBool ::= PairExpr [(‘=’ | ‘≠’ | ‘<’ | ‘>’ | ‘≤’ | ‘≥’ | ‘∈’ | ‘∉’ | ‘⊂’ | ‘⊆’ | ‘⊃’ | ‘⊇’ | ‘⊄’ | ‘⊈’ | ‘⊅’ | ‘⊉’) PairExpr]

PairExpr ::= RelSetExpr
  | PairExpr ‘↦’ RelSetExpr

RelSetExpr ::= SetExpr [(‘↔’ | ‘⇹’ | ‘↮’ | ‘⇻’ | ‘⇸’ | ‘→’ | ‘⤔’ | ‘↣’ | ‘⤖’ | ‘↠’ | ‘⤀’) RelSetExpr]

SetExpr ::= IntervalExpr [‘◁’ | ‘⩤’ | ‘▷’ | ‘⩥’] IntervalExpr
  | IntervalExpr {‘∪’ IntervalExpr}
  | IntervalExpr {‘×’ IntervalExpr}
  | IntervalExpr {‘⊕’ IntervalExpr}
  | IntervalExpr {‘∘’ IntervalExpr}
  | IntervalExpr {‘∩’ IntervalExpr}
  | IntervalExpr {‘\’ IntervalExpr}
  | IntervalExpr {‘∥’ IntervalExpr}

IntervalExpr ::= ArithmeticExpr [‘..’ ArithmeticExpr]

ArithmeticExpr ::= Term
  | ArithmeticExpr (‘+’ | ‘-’) Term

Term ::= Factor
  | Term (‘*’ | ‘/’ | ‘div’ | ‘mod’) Factor

Factor ::= [(‘+’ | ‘-’)] Power

Power ::= Primary [‘^’ Factor | [‘−’](‘$INTEGER$’ | ‘$FLOAT$’)]

Primary ::= Atom {AtomFollow}

AtomFollow ::= ‘.’ IDENTIFIER
  | ‘(’ Expr {‘,’ Expr} ‘)’
  | ‘[’ Expr {‘,’ Expr } ‘]’

Atom ::= INTEGER | FLOAT | STRING | BOOLEAN | IDENTIFIER
  | Set | Sequence | Bag | Tuple
  | ‘(’ Expr ‘)’

Set ::= ‘{’ CollectionBody ‘}’

Bag ::= ‘⟦’ CollectionBody ‘⟧’

Sequence ::= ‘⟨’ CollectionBody ‘⟩’

Tuple ::= ‘(’ ‘)’ | ‘(’ [NEWLINE INDENT] Expr ‘,’ [NEWLINE [INDENT]] [EnumerationBody] ‘)’

CollectionBody ::= QuantificationBody
  | EnumerationBody

EnumerationBody ::= [NEWLINE INDENT] Expr {‘,’ [NEWLINE [INDENT]] Expr} [NEWLINE DEDENT]

CompoundStmt ::= IfStmt | ForStmt | WhileStmt
  | RecordStmt | ProcedureStmt

IfStmt ::= ‘if’ Predicate ‘:’ Block [ElseStmt]

ElseStmt ::= ‘else :’ Block
  | ‘else if’ Predicate ‘:’ Block [ElseStmt]

ForStmt ::= ‘for’ IdentList ‘∈’ Expr ‘:’ Block

WhileStmt ::= ‘while’ Expr ‘:’ Block

RecordStmt ::= '`record`' IDENTIFIER '`:`' NEWLINE INDENT (
    '`skip`' | TypedName {('`,`' [NEWLINE] | NEWLINE) TypedName }
) NEWLINE DEDENT

ProcedureStmt ::= {'`trait`' Expr NEWLINE} '`procedure`' IDENTIFIER '`(`' [TypedName {'`,`' TypedName }] '`)`' '$\rightarrow$'
    TypeExpr '`:`' Block

Block ::= SimpleStatements
  | NEWLINE INDENT Statements DEDENT

TypedName ::= IDENTIFIER ['`:`' TypeExpr]

TypeExpr ::= IDENTIFIER {'`.`' IDENTIFIER} ['`[`' TypeExpr {'`,`' TypeExpr}'`]`']
  | '`(`' '`)`'
  | '`(`' TypeExpr '`,`' [TupleTypeExprBody] '`)`'

TupleTypeExprBody ::= TypeExpr {'`,`' TypeExpr}

ControlFlowStmt ::= '`return`' [Expr]
  | '`break`'
  | '`continue`'
  | '`skip`'

ImportStmt ::= '`import`' STRING
  | '`from`' STRING '`import`' ImportList

ImportList ::= '`*`'
  | FlatTupleIdentifier

FlatTupleIdentifier ::= IDENTIFIER {'`,`' IDENTIFIER }
  | '`(`' IDENTIFIER {'`,`' IDENTIFIER } '`)`'

2.3 Static Analysis - Type System

Let Γ be the type environment set consisting of elements either $identifier : Type$ or $Type$. The typing rules have been organized in distinct categories for readability. The notation $Type[x, y, z]$ denotes refinements on $Type$ either through instantiating parametric (generic) types or restricting values by way of `trait` clauses.

Type refinements may be applicable to types with certain traits (ex., min/max values for ordered types, definedness of expressions with certain values, set sizes).

2.3.1 Base Environment Interactions

Includes: Mechanisms for adding/extracting variables from the type environment.

$$\frac{}{\varnothing \vdash \diamond} \quad \text{(Env } \varnothing\text{)}$$

$$\frac{\Gamma \vdash A \qquad I \notin dom(\Gamma)}{\Gamma, I : A \vdash \diamond} \quad \text{(Env } I\text{)}$$

$$\frac{\Gamma_1, I : A, \Gamma_2 \vdash \diamond}{\Gamma_1, I : A, \Gamma_2 \vdash I : A} \quad \text{(Fetch Identifier)}$$

2.3.2 Subtype Interactions

Includes: Generic subtyping rules.

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \leq A} \quad \text{(Reflexive Subtype)}$$

$$\frac{\Gamma \vdash A \leq B \quad \Gamma \vdash B \leq C}{\Gamma \vdash A \leq C} \quad \text{(Transitive Subtype)}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash A \leq B}{\Gamma \vdash a : B} \quad \text{(Subsumption)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash Any} \quad \text{(Top Type)}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \leq Any} \quad \text{(Sub Top Type)}$$

$$\frac{\Gamma \vdash A' \leq A \quad \Gamma \vdash B \leq B'}{\Gamma \vdash A \to B \leq A' \to B'} \quad \text{(Sub Function)}$$

$$\frac{\Gamma \vdash A \leq B}{\Gamma \vdash set[A] \leq set[B]} \quad \text{(Sub Set)}$$

$$\frac{\Gamma \vdash A_1 \leq B_1 \quad \Gamma \vdash A_2 \leq B_2}{\Gamma \vdash A_1 \times A_2 \leq B_1 \times B_2} \quad \text{(Sub Product)}$$

$$\frac{\Gamma \vdash A_1 \leq B_1 ... A_n \leq B_n \quad \Gamma \vdash A_{n+1} ... A_{n+m}}{\Gamma \vdash Record[I_1 : A_1, ..., I_n : A_n, I_{n+1} : A_{n+1}, ... I_{n+m} : A_{n+m}] \leq Record[I_1 : B_1, ..., I_n : B_n]} \quad \text{(Sub Record)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash T[\text{with } x] \leq T} \quad \text{(Type Refinement)}$$

2.3.3 The Set Type

Includes: Universal type; everything is a set. The empty set is a part of every type (aside from primitives).

$$\frac{\Gamma \vdash T}{\Gamma \vdash set[T]} \quad \text{(Powerset)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash EmptySet} \quad \text{(Emptyset)}$$

$$\frac{\Gamma \vdash EmptySet, set[T]}{EmptySet \leq set[T]} \quad \text{(Emptyset Bottom)}$$

$$\frac{\Gamma \vdash e_1, ..., e_n : A}{\Gamma \vdash \{\, e_1, ..., e_n \,\} : set[A]} \quad \text{(Set Enumeration)}$$

2.3.4 Primitive Types

Includes: Basic types built in to the language, like numbers (int and float), strings, and booleans. Although these could all be represented as sets, it is useful to distinguish these common types from other collections or user-defined types.

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash bool} \quad \text{(Primitives - bool)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash int} \quad \text{(Primitives - int)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash float} \quad \text{(Primitives - float)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash str} \quad \text{(Primitives - str)}$$

$$\frac{\Gamma \vdash int}{\Gamma \vdash nat \leq int[\text{with } min = 0]} \quad \text{(Nat from int)}$$

$$\frac{\Gamma \vdash int, float}{int \leq float} \quad \text{(Int from float)}$$

2.3.5 Collection Types

Includes: Sequences, relations, enums (frozen sets), and bags (multisets); Syntactic sugar for abstract data types defined in terms of sets.

$$\frac{\Gamma \vdash T_1, ..., T_n}{\Gamma \vdash tuple[T_1, ..., T_n] \quad tuple[T_1, ..., T_n] = T_1 \times ... \times T_n} \quad \text{(Tuples)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash () : EmptyTuple} \quad \text{(Empty Tuple)}$$

$$\frac{\Gamma \vdash EmptyTuple, T}{\Gamma \vdash EmptyTuple \leq tuple[T]} \quad \text{(Empty Tuple Bottom)}$$

$$\frac{\Gamma \vdash e_1 : A_1, ..., e_n : A_n}{\Gamma \vdash (e_1, ..., e_n) : tuple[A_1, ..., A_n]} \quad \text{(Tuple Enumeration)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \leftrightarrow T_2 \quad T_1 \leftrightarrow T_2 = set[tuple[T_1, T_2]]} \quad \text{(Relation Type)}$$

$$\frac{\Gamma \vdash T, nat}{\Gamma \vdash bag[T] \leq T \nrightarrow nat} \quad \text{(Bag Type)}$$

$$\frac{\Gamma \vdash e_1, ..., e_n : A}{\Gamma \vdash [\![e_1, ..., e_n]\!] : bag[A]} \quad \text{(Bag Enumeration)}$$

$$\frac{\Gamma \vdash T, nat}{\Gamma \vdash sequence[T] \leq nat \nrightarrow T} \quad \text{(Sequence Type)}$$

$$\frac{\Gamma \vdash e_1, ..., e_n : A}{\Gamma \vdash \langle e_1, ..., e_n \rangle : sequence[A]} \quad \text{(Sequence Enumeration)}$$

$$\frac{\Gamma \vdash s_1, ..., s_n : str \quad A = \{ s_1, ..., s_n \} \quad immutable(A)}{\Gamma \vdash enum[A] \leq set[A, \text{with } immutable]}$$ (Enum from static set)

2.3.6 Relation Subtypes

Includes: Relation subtypes examine four key properties of relations: totality (t), surjectivity (t_r), one-to-manyness ($1-$), and many-to-oneness ($1-_r$). Syntactic sugar to produce refined relations with these properties follows the definitions presented in Event-B. See Section 5.1.3 for more details.

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nrightarrow T_2 \quad T_1 \nrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t]}$$ (Relation Subtype - Total Relation)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nleftrightarrow T_2 \quad T_1 \nleftrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t_r]}$$ (Relation Subtype - Surjective Relation)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nrightarrow T_2 \quad T_1 \nrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t, \text{ with } t_r]}$$ (Relation Subtype - Total Surjective Relation)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nrightarrow T_2 \quad T_1 \nrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } 1-]}$$ (Relation Subtype - Partial Function)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t, \text{ with } 1-]}$$ (Relation Subtype - Total Function)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrowtail T_2 \quad T_1 \rightarrowtail T_2 = (T_1 \leftrightarrow T_2)[\text{with } 1-, \text{ with } 1-_r]}$$ (Relation Subtype - Partial Injection)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrowtail T_2 \quad T_1 \rightarrowtail T_2 = (T_1 \leftrightarrow T_2)[\text{with } t, \text{ with } 1-, \text{ with } 1-_r]}$$ (Relation Subtype - Total Injection)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \twoheadrightarrow T_2 \quad T_1 \twoheadrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t_r, \text{ with } 1-]}$$ (Relation Subtype - Partial Surjection)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \twoheadrightarrow T_2 \quad T_1 \twoheadrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t, \text{ with } t_r, \text{ with } 1-]}$$ (Relation Subtype - Total Surjection)

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrowtail T_2 \quad T_1 \rightarrowtail T_2 = (T_1 \leftrightarrow T_2)[\text{with } t, \text{ with } t_r, \text{ with } 1-, \text{ with } 1-_r]}$$ (Relation Subtype - Bijection)

2.3.7 Simple Commands

Includes: Assignment, type assignment, type refinements, simple programming language commands/statements.

$$\frac{\Gamma \vdash E : A \quad I \notin dom(\Gamma) \vee \Gamma \vdash I : A}{\Gamma \vdash I : A := E}$$ (Variable Assignment)

$$\frac{\Gamma \vdash T \quad I \notin dom(\Gamma)}{\Gamma \vdash I := T}$$ (Type Alias Assignment)

$$\frac{\Gamma \vdash I := T}{\Gamma, I \vdash \diamond \quad I = T}$$ (Type Alias)

$$\frac{\Gamma \vdash E : A, E_1 : A_1, ..., E_n : A_n \quad (I \notin dom(\Gamma) \vee \Gamma \vdash I : A)}{\Gamma \vdash I : A[\text{with } E_1 : A_1, ..., \text{ with } E_n : A_n] := E} \quad \text{(Refined Variable Assignment)}$$

$$\frac{\Gamma \vdash T, E_1 : A_1, ..., E_n : A_n \quad I \notin dom(\Gamma)}{\Gamma \vdash I := T[\text{with } E_1 : A_1, ..., \text{ with } E_n : A_n]} \quad \text{(Refined Type Alias)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{break} : C} \quad \text{(Command - break)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{continue} : C} \quad \text{(Command - continue)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{skip} : C} \quad \text{(Command - skip)}$$

$$\frac{\Gamma \vdash E : A}{\Gamma \vdash (\text{return } E) : C} \quad \text{(Command - return)}$$

2.3.8 Quantification Expressions

Includes: Set, sequence, relation, and bag comprehensions, in addition to (set based) lambda expressions. Comprehension variables may be bound through either an $Identifier$ or an arbitrarily nested tuple of $Identifiers$

$$\frac{\Gamma, I_1 : T_1, ..., I_n : T_n \vdash E : A, P : bool \quad I_i \notin dom(\Gamma) \quad i \neq j \implies I_i \neq I_j}{\Gamma \vdash (\Lambda I_1, ..., I_n P E) : (T_1, ..., T_n) \nrightarrow A} \quad \text{(Lambda Expression)}$$

$$\frac{\Gamma \vdash G : Generator[I] \quad \Gamma, I \vdash E : A}{\Gamma \vdash (G \mid E) : Quantifier[A]} \quad \text{(Quantifier)}$$

$$\frac{\Gamma \vdash Q_i : Quantifier[A]}{\Gamma \vdash (Q_1 \text{`...`} Q_n) : Quantifier[A]} \quad \text{(Branching Quantifier)}$$

$$\frac{\Gamma \vdash G_i : Generator[T_i]}{\Gamma \vdash G_1, ..., G_n : Generator[(T_1, ..., T_n)]} \quad \text{(Generator nest)}$$

$$\frac{\Gamma \vdash S : set[T], P : bool \quad I = x_1 : T_1, ..., x_n : T_n \quad \Gamma \vdash structuralMatch(I, T)}{\Gamma \vdash (I \in S \cdot P) : Generator[T]} \quad \text{(Generator)}$$

$$\frac{\Gamma \vdash T \quad x \notin dom(\Gamma)}{\Gamma \vdash structuralMatch(x, T)} \quad \text{(Structural Match)}$$

$$\frac{\Gamma \vdash structuralMatch(x_1, T_1), ..., structuralMatch(x_n, T_n)}{\Gamma \vdash structuralMatch((x_1, ..., x_n), (T_1, ..., T_n))} \quad \text{(Structural Match with Tuple)}$$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash \{\, Q \,\} : set[A]} \quad \text{(Set Comprehension)}$$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash [\![\, Q \,]\!] : bag[A]} \quad \text{(Bag Comprehension)}$$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash [\, Q \,] : sequence[A]} \quad \text{(Sequence Comprehension)}$$

$$\frac{\Gamma, i : A \vdash Q : Quantifier[A] \quad \Gamma \vdash Id : A \quad i \notin dom(\Gamma)}{\Gamma \vdash (fold\ i := Id : Q) : A} \quad \text{(Fold)}$$

$$\frac{\Gamma \vdash G : IterGenerator[I], x_i : T_i, \text{block } C}{\Gamma \vdash (G \to (x_1, ..., x_n) \mid C) : IterQuantifier[(T_1, ..., T_n)]} \quad \text{(Iter Quantifier)}$$

$$\frac{\Gamma \vdash G : Generator[I], y_i : T_i \qquad x_i \notin dom(\Gamma)}{\Gamma \vdash (G : x_1 := y_1; ...; x_n := y_n) : IterGenerator[I]} \quad \text{(Iter Generator)}$$

$$\frac{\Gamma \vdash Q : IterQuantifier[A]}{\Gamma \vdash (iter\ Q) : A} \quad \text{(Iter)}$$

Let $\otimes = \{ \bigcup, \bigcap \}$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash (\otimes Q) : A} \quad \text{(General Quantifiers)}$$

Let $\otimes = \{ \forall, \exists \}$

$$\frac{\Gamma \vdash Q : Quantifier[bool]}{\Gamma \vdash (\otimes Q) : bool} \quad \text{(Bool Quantifiers)}$$

Let $\otimes = \{ \sum, \prod \}$

$$\frac{\Gamma \vdash Q : Quantifier[T] \qquad \Gamma \vdash T \in \{ int, nat, float \}}{\Gamma \vdash (\otimes Q) : T} \quad \text{(Numeric Quantifiers)}$$

2.3.9 Boolean Operations

Includes: Common predicate logic operations (equivalence, implication, and, or, etc.).
Let $\otimes = \{ \equiv, \not\equiv, \implies, \wedge, \vee \}$

$$\frac{\Gamma \vdash b_1, b_2 : bool}{\Gamma \vdash b_1 \otimes b_2 : bool} \quad \text{(Binary Boolean Operations)}$$

$$\frac{\Gamma \vdash b_1 : bool}{\Gamma \vdash \neg b_1 : bool} \quad \text{(Boolean Operations - Negation)}$$

2.3.10 Equality and Membership

Includes: Equality and ordering comparisons, set membership.
Let $\otimes = \{ =, \neq \}$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Equals)}$$

Let $\otimes = \{ <, \leq, >, \geq \}$

$$\frac{\Gamma \vdash a_1, a_2 : T \qquad T \in \{ int, float, nat \}}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Ordering Operators)}$$

Let $\otimes = \{ \in, \notin \}$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : set[T_2]}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Set Membership)}$$

2.3.11 Set Operations

Includes: Subset comparisons, union, intersection, cartesian product, etc. Also includes Maplet and Range.

Let $\otimes = \{ \subset, \subseteq, \supset, \supseteq, \not\subset, \not\subseteq, \not\supset, \not\supseteq \}$

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : set[T_2]}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Set Ordering Operations)}$$

Let $\otimes = \{ \cup, \cap, \setminus \}$

$$\frac{\Gamma \vdash a_1, a_2 : set[T]}{\Gamma \vdash a_1 \otimes a_2 : set[T]} \quad \text{(Set Operations)}$$

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : set[T_2]}{\Gamma \vdash a_1 \times a_2 : T_1 \leftrightarrow T_2} \quad \text{(Cartesian Product)}$$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2}{\Gamma \vdash a_1 \mapsto a_2 : tuple[T_1, T_2]} \quad \text{(Maplet)}$$

$$\frac{\Gamma \vdash a_1, a_2 : int}{\Gamma \vdash a_1 \mathinner{..} a_2 : set[int, min = a_1, max = a_2]} \quad \text{(Numerical Range)}$$

$$\frac{\Gamma \vdash a : T}{\Gamma \vdash \mathbb{P}(a) : set[T]} \quad \text{(Set Operations - Powerset)}$$

2.3.12 Bag Operations

Includes: Bag union, bag intersection, etc.

Let $\otimes = \{ \uplus, \cap, +, - \}$

$$\frac{\Gamma \vdash a_1, a_2 : bag[T]}{\Gamma \vdash a_1 \otimes a_2 : bag[T]} \quad \text{(Bag Operations)}$$

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : bag[T_1]}{\Gamma \vdash a[b] : bag[T_2] \quad a[b] = a^{-1} \circ b} \quad \text{(Bag Operations - Image)}$$

2.3.13 Relation Operations

Includes: Relation overriding, composition, domain/range manipulation. See Section 5.1.3 for changes in relational subtypes after applying operations.

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : T_1}{\Gamma \vdash a(b) : T_2} \quad \text{(Relation Operations - Function Call)}$$

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : set[T_1]}{\Gamma \vdash a[b] : set[T_2]}$$ (Relation Operations - Image)

$$\frac{\Gamma \vdash a_1, a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \oplus a_2 : T_1 \leftrightarrow T_2}$$ (Relation Operations - Overriding)

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : T_2 \leftrightarrow T_3}{\Gamma \vdash a_1 \circ a_2 : T_1 \leftrightarrow T_3}$$ (Relation Operations - Composition)

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \lhd a_2 : T_1 \leftrightarrow T_2}$$ (Relation Operations - Domain Restriction)

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \lhd\!\!\!- a_2 : T_1 \leftrightarrow T_2}$$ (Relation Operations - Domain Subtraction)

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : set[T_2]}{\Gamma \vdash a_1 \rhd a_2 : T_1 \leftrightarrow T_2}$$ (Relation Operations - Range Restriction)

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : set[T_2]}{\Gamma \vdash a_1 \rhd\!\!\!- a_2 : T_1 \leftrightarrow T_2}$$ (Relation Operations - Range Subtraction)

2.3.14 Sequence Operations

Includes: Concatenation

$$\frac{\Gamma \vdash a_1, a_2 : sequence[T]}{\Gamma \vdash a_1 +\!\!+ a_2 : sequence[T]}$$ (Sequence Operations - Concatenation)

2.3.15 Numerical Operations

Includes: Addition, subtraction, multiplication, etc. on natural numbers, integers, and floats.

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{ int, nat \}}{\Gamma \vdash a_1 \text{ div } a_2 : max(T_1, T_2)}$$ (Integer Operations - Division)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{ int, nat \}}{\Gamma \vdash a_1 \text{ mod } a_2 : max(T_1, T_2)}$$ (Integer Operations - Modulo)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{ int, nat, float \}}{\Gamma \vdash a_1 + a_2 : max(T_1, T_2)}$$ (Numerical Operations - Addition)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{ int, nat, float \}}{\Gamma \vdash a_1 - a_2 : max(T_1, T_2, int)}$$ (Numerical Operations - Subtraction)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{ int, nat, float \}}{\Gamma \vdash a_1 / a_2 : float}$$ (Numerical Operations - Floating Division)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{ int, nat, float \}}{\Gamma \vdash a_1 * a_2 : max(T_1, T_2)}$$ (Numerical Operations - Multiplication)

$$\frac{\Gamma \vdash a_1 : T \quad T \in \{ int, nat, float \}}{\Gamma \vdash -a_1 : max(int, T)}$$ (Numerical Operations - Negation)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{\, int, nat, float \,\}}{\Gamma \vdash a_1 \hat{} \ a_2 : max(T_1, T_2)}$$ (Numerical Operations - Exponentiation)

2.3.16 Record Types

Includes: Types for named tuples.

$$\frac{\Gamma \vdash a : Record[I : A, ...]}{\Gamma \vdash a.I : A}$$ (Records - Access)

$$\frac{\Gamma \vdash A_1, ..., A_n \quad \forall\, I_i, I_j \mid I_i \neq I_j}{\Gamma \vdash Record[I_1 : A_1, ..., I_n : A_n]}$$ (Records - Type Definition)

$$\frac{\Gamma \vdash a_1 : A_1, ..., a_n : A_n}{\Gamma \vdash record(a_1 = A_1, ..., a_n = A_n) : Record[a_1 : A_1, ..., a_n : A_n]}$$ (Records - Initialization)

2.3.17 Compound Commands

Includes: If, While, For, and Import statements, along with Procedure definitions and calls.

$$\frac{\Gamma \vdash C_1, C_2}{\Gamma \vdash C_1 \ \backslash \text{n}\ C_2}$$ (Command - Composition)

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{import } M : C}$$ (Command - Valid Import Module)

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{from } M \text{ import } I_1, ..., I_n : C}$$ (Command - Valid Import Names)

$$\frac{\Gamma \vdash \text{import } M \text{ (with identifiers and types } I_1 : A_1, ..., I_n : A_n)}{\Gamma, M = record(I_1 = A_1, ..., I_n = A_n) \vdash \diamond}$$ (Command - Import Module)

$$\frac{\Gamma \vdash \text{from } M \text{ import } I_1, ..., I_n \text{ (with types } A_1, ..., A_n)}{\Gamma, I_1 : A_1, ..., I_n : A_n \vdash \diamond}$$ (Command - Import Names)

$$\frac{\Gamma, I : procedure[A_1, ..., A_n, R], x_1 : A_1, ..., x_n : A_n \vdash C : R \quad I, x_1, ..., x_n \notin dom(\Gamma) \quad i \neq j \implies x_i \neq x_j}{\Gamma \vdash \ \text{procedure } I = C : procedure[A_1, ..., A_n, R]}$$ (Command - Procedure Definition)

$$\frac{\Gamma \vdash procedure[A_1, ..., A_n, R], a_1 : A_1, ..., a_n : A_n}{\Gamma \vdash I(a_1, ..., a_n) : R}$$ (Command - Procedure Call)

$$\frac{\Gamma \vdash b : bool, \text{block } C}{\Gamma \vdash \text{if } b : C}$$ (Command - If)

$$\frac{\Gamma \vdash b : bool, \text{block } C_1, \text{block } C_2}{\Gamma \vdash \text{if } b : C_1 \text{ else} : C_2}$$ (Command - If Else)

$$\frac{\Gamma \vdash b : bool, \text{block } C}{\Gamma \vdash \text{while } b : C}$$ (Command - While)

$$\frac{\Gamma \vdash G : set[T] \quad \Gamma, i : T \vdash C \quad i \notin dom(\Gamma)}{\Gamma \vdash \text{for } i \in G : C}$$ (Command - For)

$$\frac{\Gamma \vdash D \therefore \Gamma' \qquad \Gamma' \vdash C}{\Gamma \vdash \text{block } D \text{ \textbackslash n } C} \quad \text{(Command - Block)}$$

2.3.18 Built-in Functions

Includes: Common set-related functions.

$$\frac{\Gamma \vdash T[\text{with } order \in traits(T)]}{\Gamma \vdash min : set[T] \to T} \quad \text{(Built-in - Minimum)}$$

$$\frac{\Gamma \vdash T, int}{\Gamma \vdash min : set[T] \times (T \to int) \to T} \quad \text{(Built-in - Mapped Minimum)}$$

$$\frac{\Gamma \vdash T[\text{with } order \in traits(T)]}{\Gamma \vdash max : set[T] \to T} \quad \text{(Built-in - Maximum)}$$

$$\frac{\Gamma \vdash T, int}{\Gamma \vdash max : set[T] \times (T \to int) \to T} \quad \text{(Built-in - Mapped Maximum)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash choice : set[T] \to T} \quad \text{(Built-in - Choice)}$$

$$\frac{\Gamma \vdash T_1, T_2}{\Gamma \vdash dom : T_1 \leftrightarrow T_2 \to set[T_1]} \quad \text{(Built-in - Domain)}$$

$$\frac{\Gamma \vdash T_1, T_2}{\Gamma \vdash ran : T_1 \leftrightarrow T_2 \to set[T_2]} \quad \text{(Built-in - Range)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash card : set[T] \to nat} \quad \text{(Built-in - Cardinality)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash size : bag[T] \to nat} \quad \text{(Built-in - Bag Size)}$$

$$\frac{\Gamma \vdash T \qquad T \in \{ int, float, nat \}}{\Gamma \vdash sum : set[T] \to T} \quad \text{(Built-in - Sum)}$$

$$\frac{\Gamma \vdash T_1, T_2}{\Gamma \vdash cast : type[T_2] \times T_1 \to T_2} \quad \text{(Built-in - Cast)}$$

$$\frac{\Gamma \vdash T_1, T_2, x : E}{\Gamma \vdash cast : type[T_2] \times T_1 \times set[E] \to T_2[\text{with } x : E]} \quad \text{(Built-in - Cast With } ^{8}\text{)}$$

2.4 Static Analysis - Type System Extensions with Traits

Traits extend Simile's type system by giving programmers the ability to provide additional context for the intended use of a defined variable. Traits never override Simile's set theoretic semantics; a program with no user-defined traits is assumed to be correct and functional. However, traits can influence the optimization process and concretization of abstract data structures. With user-provided context, static analysis becomes more complete and the optimizer may choose more aggressive rewrites. In a sense, traits in Simile are akin to pragmas in other languages, but closer to type refinements rather than direct compiler commands.

For example, consider a variable defining a relation:

$r : relation[int, int] := \{\ \}$

Without insight into the use of the variable r, we may be forced to concretize the abstract type conservatively. The compiler may choose the safest option: straightforward array of tuples or a bidirectional hashmap, with both concrete structures giving up time complexity or memory efficiency respectively. However, if we as domain experts know that this relation is one-to-one, we can provide such insights to the compiler directly:

$r : relation[int, int] := \{\ \}$
  trait $many_to_one$
  trait one_to_many

Concretization can then see that a hashmap is well-suited to representing the data. Even further, since r is one-to-one, we may even choose to save memory by concretizing to two arrays aligned by index. To the programmer, the important semantics remain the same; r is a relation and behaves like one for the duration of the program, but the compiler can choose to swap the detailed implementation with one better-suited for the problem.

There is one caveat to building these features into the language itself: we trust that users will not provide deliberately misleading traits. It is difficult to check that trait assignments themselves are correct or well-defined through static analysis alone, so an unknowing programmer may stumble into undefined behaviour. For example, consider:

$x : int := 3$
  trait $minimum = 1$
  trait $maximum = 4$

The compiler may see that this integer is restricted to a very small range and thus choose a smaller integer representation size. However, if the programmer then writes:

$x := 5$

This assumption breaks and does not fit into the general type. We may be able to spot such hardcoded misuses of the trait system, but if 5 was instead an I/O function, we can only provide runtime checks that slow down the system (and may turn out to be slower than just using a normal i32 type in the first place).

Given the power and danger of using traits, programs without traits by default should behave as expected. Note that this danger happens only on assignment, since traits are assumed correct for the variable to which they are assigned. When performing operations, the compiler will widen or narrow traits as is needed to remain in a good state.

2.4.1 Available Traits

The following paragraphs list and categorize identified traits.

For parameterized types that accept type arguments, traits will be propagated to the "parent" type as much as possible. For example, a $set[int]$ restricting the range of the underlying integers should have that trait specified on the outer *set* type rather than the underlying *int* type.

Base Traits Applicable to almost every object

- *Literal*: A trait containing a single literal value. If a type has this trait, it is guaranteed to be of this value.
- *Domain*: A looser form of *Literal* that carries a set of possible values. When applied, the type is restricted to (one of) the values from this *Domain*. Relations use a different definition of domain.
- *Undefined*: Intended for use by compiler analysis. Used when a value *may* be undefined. Ex. $choice(\{\ \})$.
- *Immutable*: Represents values that cannot be reassigned/modified.

Arithmetic Restricts the value of primitive types

- *Orderable*: Values of this type can be ordered. This may be useful for order-based implementations like BTrees for sets.
- *Min*: Minimum value.
- *Max*: Maximum value. Combine with the *Min* trait to create a range.

Collection Traits

- *Iterable*: All collections are iterable (they can be used as a for-loop generator).
- *Empty*: A collection with guaranteed cardinality = 0.
- *Size*: An over-estimate of a set's size. When available, Size will never under-estimate the number of elements in a collection.
- *Unique*: All entries are guaranteed to be unique. More useful for non-set collections (which are always assumed to be unique). Ex. a non-repeating sequence.
- *Total*: The elements in a set cover the entirety of the domain of the underlying element type. Relations have their own definition of totality.

Relations Represents the relational subtype properties (injectiveness, surjectiveness, etc.)

- *ManyToOne*: Elements of the domain may be repeated with different range values.
- *OneToMany*: Elements of the range may be repeated with different range values. *ManyToOne* and *OneToMany* imply a one-to-one mapping.
- *TotalOnDomain*: All elements of the domain are within the mapping.
- *TotalOnRange*: All elements of the range are within the mapping.
- *RelationalDomain*: The set of elements that may populate the relation's domain. Replacement of *Domain* for relations.
- *RelationalRange*: The set of elements that may populate the relation's range.
- *RelationalDomainMin*: Minimum of the relation domain.
- *RelationalDomainMax*: Maximum of the relation domain.
- *RelationalRangeMin*: Minimum of the relation range.
- *RelationalRangeMax*: Maximum of the relation range.

Generics

- *GenericBound*: Contains the set of types a generic may become. For example, the following generic type represents any numeric value:

  $T : type := generic$
  trait $generic_bound = int$
  trait $generic_bound = float$

Procedure

- *TreatAsExpr*: Inline this function and treat it as an expression/operator. Useful for having function-call notation act like an operator. For example:

  procedure $add(s : set[T], e : T) \to set[T]$:
    trait $treat_as_expr$
    return $s \cup \{ e \}$

2.4.2 Type-trait interactions

Some types are incompatible with some traits, and some types must have some traits. We provide tables showing compatibility between traits and types. A double-checkmark means the trait is always present - a base trait of that type.

Primitives

	none	bool	int	float	str
Immutable	✓	✓	✓	✓	✓
Literal	None	✓	✓	✓	✓
Undefined	✓	✓	✓	✓	✓
Orderable	✗	✗	✓✓	✓✓	✓✓
Min	✗	✗	✓	✓	✓
Max	✗	✗	✓	✓	✓
Domain	{ None }	{ true, false }	✓	✓	✓
Iterable	✗	✗	✗	✗	✓✓
Unique	✗	✗	✗	✗	✗
Empty	✗	✗	✗	✗	✓
Size	✗	✗	✗	✗	✓

Collections

	tuple	set	enum	relation	bag	sequence
Immutable	✓	✓	✓✓	✓	✓	✓
Literal	✓	✓	✓	✓	✓	✓
Undefined	✓	✓	✓	✓	✓	✓
Orderable	✓	✓	✓	✓	✓	✓
Min	✓	✓	✓	✗	✓	✓
Max	✓	✓	✓	✗	✓	✓
Domain	✓	✓	✓✓	✗	✓	✓
Iterable	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓
Unique	✓	✓✓	✓✓	✓	✓	✓
Empty	✓	✓	✓	✓	✓	✓
Size	✓	✓	✓✓	✓	✓	✓
Total	✓	✓	✓	✗	✓	✓

	tuple	set	enum	relation	bag	sequence
OneToMany	✗	✗	✗	✓	✓	✓
ManyToOne	✗	✗	✗	✓	✓✓	✓✓
TotalOnDomain	✗	✗	✗	✓	✗	✗
TotalOnRange	✗	✗	✗	✓	✗	✗
RelationalDomain	✗	✗	✗	✓	✗	✗
RelationalRange	✗	✗	✗	✓	✗	✗
RelationalDomainMin	✗	✗	✗	✓	✗	✗
RelationalDomainMax	✗	✗	✗	✓	✗	✗
RelationalRangeMin	✗	✗	✗	✓	✗	✗
RelationalRangeMax	✗	✗	✗	✓	✗	✗
GenericBound	✗	✗	✗	✗	✗	✗
TreatAsExpr	✗	✗	✗	✗	✗	✗

Other

	record	procedure	type (generic)
Immutable	✓	✗	✓
Literal	✓	✗	✓
Undefined	✓	✓	✓
Orderable	✗	✗	✓
Min	✗	✗	✓
Max	✗	✗	✓
Domain	✗	✗	✓
Iterable	✓✓	✗	✓
Unique	✗	✗	✓
Empty	✗	✗	✓
Size	✓✓	✗	✓
Total	✗	✗	✓
OneToMany	✗	✗	✓
ManyToOne	✓✓	✗	✓
TotalOnDomain	✓✓	✗	✓
TotalOnRange	✗	✗	✓
RelationalDomain	✓✓(field names)	✗	✓
RelationalRange	✗	✗	✓
RelationalDomainMin	✗	✗	✓
RelationalDomainMax	✗	✗	✓
RelationalRangeMin	✗	✗	✓
RelationalRangeMax	✗	✗	✓
GenericBound	✗	✗	✓
TreatAsExpr	✗	✓	✓

2.4.3 Trait-trait Interactions

Traits can interact with other traits outside of types. Universal interactions, which don't follow a type or its operations, generally fall into two categories:

- Derivable traits. Often, the jurisdiction of traits will overlap with one another: a Literal trait implies a *Domain*, and we can derive the *Min*/*Max* traits from an *Orderable Domain*. These will be automatically derived by the compiler when needed.
- Incompatible traits. Interactions between these traits are unresolvable automatically and would leave the trait system in a bad state if left alone. To handle incompatible traits safely, we refuse to compile the program until the issue is resolved. Automatically derived traits should never become incompatible (only user-defined traits clashing with user/automatic traits).

Most traits are simple flags, as in, "does this type have this property?". In the following type (or rather, trait) rules, trait names used with boolean operations should be interpreted as truthy (it is present). A type can only have one of each trait to prevent ambiguity or clashing.

Derivable Traits

$$\frac{Domain = \{\ \}}{\neg Domain} \quad \text{(Domain Empty)}$$

$$\frac{Literal \quad Literal.type \in \{\, bool, int, float, string, none, tuple \,\} \quad \neg Domain}{Domain := \{\, Literal \,\}} \quad \text{(Literal implies a Domain (under primitive))}$$

$$\frac{Literal \quad Literal.type \in \{\, set, relation, bag, sequence \,\} \quad \neg Domain}{Domain := Literal} \quad \text{(Literal implies a Domain (under collection))}$$

$$\frac{Literal \quad Domain}{Domain := Domain \cup Literal} \quad \text{(Literal within Domain)}$$

$$\frac{Domain \quad Orderable \quad \neg Min}{Min := min(Domain)} \quad \text{(Orderable Domain without Min)}$$

$$\frac{Domain \quad Orderable \quad Min}{Min := min(Min, Domain)} \quad \text{(Orderable Domain with Min)}$$

$$\frac{Domain \quad Orderable \quad \neg Max}{Max = max(Domain)} \quad \text{(Orderable Domain without Max)}$$

$$\frac{Domain \quad Orderable \quad Max}{Max = max(Max, Domain)} \quad \text{(Orderable Domain with Max)}$$

$$\frac{Min}{Orderable} \quad \text{(Min implies Order)}$$

$$\frac{Max}{Orderable} \quad \text{(Max implies Order)}$$

$$\frac{Unique \quad Size = len(Domain) \quad \neg Empty}{Total} \quad \text{(Full set)}$$

$$\frac{Size = 0}{Empty} \quad \text{(Empty Size)}$$

$$\frac{Size \neq 0}{\neg Empty} \quad \text{(Non-Empty Size)}$$

$$\frac{Size}{Iterable} \quad \text{(Size implies Iterable)}$$

$$\frac{Literal \quad Orderable \quad \neg Min}{Min = Literal} \quad \text{(Orderable Literal is Min)}$$

$$\frac{Literal \quad Orderable \quad \neg Max}{Max = Literal} \quad \text{(Orderable Literal is Max)}$$

Incompatible Traits

2.4.4 Operation-Trait Interactions

Operations on *Literal* traits will always produce a *Literal* where possible, following the effects of applying the function on values. For conciseness, we omit all *Literal* interactions and instead refer to the semantics of the operations themselves. Further, we use a shorthand for defining traits on types: $T[< trait >]$ is the same as saying:

$T : type := generic$
trait $< trait >$

General notes:

- Any operation with an undefined value will have an undefined result.
- Any operations with immutable inputs will have an immutable output. If multi-arg operations have at least one mutable input, the output will also be mutable (unless immutability appears in the traits below).

Base

Equals Not equals is just not applied to equals.

$$\frac{a : X[literal = x] \quad b : Y[literal = y]}{(a = b) : Bool[literal = (X = Y \wedge x = y)]} \quad \text{(Literal)}$$

Bool

Not

$$\frac{p : Bool[literal = x]}{(\neg p) : Bool[literal = (\neg x)]} \quad \text{(Literal)}$$

Equivalent Not equivalent is just not applied to the equivalence operation.

$$\frac{a : Bool[literal = x] \quad b : Bool[literal = y]}{(a \equiv b) : Bool[literal = (x = y)]} \quad \text{(Literal)}$$

Implies

$$\frac{a : Bool[literal = x] \quad b : Bool[literal = y]}{(a \implies b) : Bool[literal = (x \implies y)]} \quad \text{(Literal)}$$

And

$$\frac{a : Bool[literal = x] \quad b : Bool[literal = y]}{(a \wedge b) : Bool[literal = (x \wedge y)]} \quad \text{(Literal)}$$

$$\frac{a : Bool[undefined] \quad b : Bool[literal = false]}{(a \wedge b) : Bool[literal = false]} \quad \text{(Undefined with false)}$$

$$\frac{a : Bool[undefined] \quad b : Bool[literal = true]}{(a \wedge b) : Bool[undefined]} \quad \text{(Undefined with true)}$$

$$\frac{a : Bool[undefined] \quad b : Bool}{(a \wedge b) : Bool} \quad \text{(Undefined is not guaranteed)}$$

Or

$$\frac{a : Bool[literal = x] \quad b : Bool[literal = y]}{(a \vee b) : Bool[literal = (x \vee y)]} \quad \text{(Literal)}$$

$$\frac{a : Bool[undefined] \quad b : Bool[literal = false]}{(a \vee b) : Bool[undefined]} \quad \text{(Undefined with false)}$$

$$\frac{a : Bool[undefined] \quad b : Bool[literal = true]}{(a \vee b) : Bool[literal = true]} \quad \text{(Undefined with true)}$$

$$\frac{a : Bool[undefined] \quad b : Bool}{(a \vee b) : Bool} \quad \text{(Undefined is not guaranteed)}$$

Arithmetic

Less than Opposite for greater than.

$$\frac{a : T[literal = x, orderable] \quad b : T[literal = y, orderable] \quad x < y}{(a < b) : Bool[literal = (x < y)]} \quad \text{(Literal)}$$

$$\frac{a : T[max = x] \quad b : T[min = y] \quad x < y}{(a < b) : Bool[literal = true]} \quad \text{(Max-min literal true)}$$

$$\frac{a : T[min = x] \quad b : T[max = y] \quad x \geq y}{(a < b) : Bool[literal = false]} \quad \text{(Max-min literal false)}$$

Less than or equals Opposite for greater than or equals.

$$\frac{a : T[literal = x, orderable] \quad b : T[literal = y, orderable] \quad x \leq y}{(a \leq b) : Bool[literal = (x \leq y)]} \quad \text{(Literal)}$$

$$\frac{a : T[max = x] \quad b : T[min = y] \quad x \leq y}{(a \leq b) : Bool[literal = true]} \quad \text{(Less than or equal with max-min)}$$

$$\frac{a : T[min = x] \quad b : T[max = y] \quad x > y}{(a \leq b) : Bool[literal = false]} \quad \text{(Less than or equal with min-max)}$$

Negate

$$\frac{p : T[literal = x] \quad T \in \{\, Int, Float \,\}}{(-p) : T[literal = (-x)]} \text{ (Literal)}$$

$$\frac{p : T[min = x] \quad T \in \{\, Int, Float \,\}}{(-p) : T[max = (-x)]} \text{ (Min)}$$

$$\frac{p : T[max = x] \quad T \in \{\, Int, Float \,\}}{(-p) : T[min = (-x)]} \text{ (Max)}$$

$$\frac{p : T[domain = s] \quad T \in \{\, Int, Float \,\}}{(-p) : T[domain = \{\, x \in s \mid -x \,\}]} \text{ (Domain)}$$

Int divide

$$\frac{a : Int[literal = x] \quad b : Int[literal = y]}{(a \text{ div } b) : T[literal = (\lfloor x/y \rfloor)]} \text{ (Literal)}$$

$$\frac{a : Int[literal = 0] \quad b : Int[0 \notin domain \vee max < 0 \vee min > 0]}{(a \text{ div } b) : T[literal = 0]} \text{ (Literal zero)}$$

$$\frac{a : Int[min = x] \quad b : Int[literal = y] \quad x \geq 0}{(a \text{ div } b) : T[min = (\lfloor x/y \rfloor)]} \text{ (Min and literal)}$$

$$\frac{a : Int[max = x] \quad b : Int[literal = y] \quad x \geq 0}{(a \text{ div } b) : T[max = (\lfloor x/y \rfloor)]} \text{ (Max and literal)}$$

$$\frac{a : Int[literal = x] \quad b : Int[min = y] \quad x > 0 \quad y > 0}{(a \text{ div } b) : T[max = (\lfloor x/y \rfloor)]} \text{ (Literal and min)}$$

$$\frac{a : Int[literal = x] \quad b : Int[max = y] \quad x > 0 \quad y < 0}{(a \text{ div } b) : T[min = (\lfloor x/y \rfloor)]} \text{ (Literal and max)}$$

$$\frac{a : Int[literal = x] \quad b : Int[min = y] \quad x < 0 \quad y > 0}{(a \text{ div } b) : T[min = (\lfloor x/y \rfloor)]} \text{ (Negative literal and min)}$$

$$\frac{a : Int[literal = x] \quad b : Int[max = y] \quad x < 0 \quad y < 0}{(a \text{ div } b) : T[max = (\lfloor x/y \rfloor)]} \text{ (Negative literal and max)}$$

Modulo

$$\frac{a : Int[literal = x] \quad b : Int[literal = y]}{(a \bmod b) : T[literal = (x \bmod y)]} \text{ (Literal)}$$

$$\frac{a : Int \quad b : Int[max = x] \quad x > 0}{(a \bmod b) : T[max = x, min = 0]} \text{ (Positive literal)}$$

$$\frac{a : Int \quad b : Int[min = x] \quad x < 0}{(a \bmod b) : T[min = x, max = 0]} \text{ (Negative literal)}$$

Add And commutative versions.

$$\frac{a : T[literal = x] \quad b : T[literal = y] \quad T \in \{\, Int, Float \,\}}{(a + b) : T[literal = (x + y)]} \text{ (Literal)}$$

$$\frac{a : T[literal = x] \quad b : T[min = y] \quad T \in \{\, Int, Float \,\}}{(a + b) : T[min = (x + y)]} \text{ (Literal and min)}$$

$$\frac{a : T[literal = x] \quad b : T[max = y] \quad T \in \{\, Int, Float \,\}}{(a + b) : T[max = (x + y)]} \text{ (Literal and max)}$$

$$\frac{a : T \quad b : T[min = y] \quad T \in \{\, Int, Float \,\}}{(a + b) : T[min = y]} \text{ (Min)}$$

$$\frac{a : T \quad b : T[max = y] \quad T \in \{\, Int, Float \,\}}{(a + b) : T[max = y]} \text{ (Max)}$$

$$\frac{a : T[domain = m] \quad b : T[domain = n] \quad T \in \{\, Int, Float \,\}}{(a + b) : T[domain = map(sum, m \times n)]} \text{ (Domain widens)}$$

Subtract

$$\frac{a : T[literal = x] \quad b : T[literal = y] \quad T \in \{\, Int, Float \,\}}{(a - b) : T[literal = (x - y)]} \text{ (Literal)}$$

$$\frac{a : T[literal = x] \quad b : T[min = y] \quad T \in \{\, Int, Float \,\}}{(a - b) : T[max = (x - y)]} \text{ (Literal and min)}$$

$$\frac{a : T[literal = x] \quad b : T[max = y] \quad T \in \{\, Int, Float \,\}}{(a - b) : T[min = (x - y)]} \text{ (Literal and max)}$$

$$\frac{a : T[min = x] \quad b : T[literal = y] \quad T \in \{\, Int, Float \,\}}{(a - b) : T[min = (x - y)]} \text{ (Min and literal)}$$

$$\frac{a : T[max = x] \quad b : T[literal = y] \quad T \in \{\, Int, Float \,\}}{(a - b) : T[max = (x - y)]} \text{ (Max and literal)}$$

$$\frac{a : T[domain = m] \quad b : T[domain = n] \quad T \in \{\, Int, Float \,\}}{(a - b) : T[domain = map(sum, m \times map(-, n))]} \text{ (Domain widens)}$$

Divide

$$\frac{a : Int[literal = x] \quad b : Int[literal = y]}{(a/b) : T[literal = (x/y)]} \text{ (Literal)}$$

$$\frac{a : Int[literal = 0] \quad b : Int[0 \notin domain \vee max < 0 \vee min > 0] \quad x \geq 0}{(a/b) : T[literal = 0]} \text{ (Literal zero)}$$

$$\frac{a : Int[min = x] \quad b : Int[literal = y] \quad x \geq 0}{(a/b) : T[min = (x/y)]} \text{ (Min and literal)}$$

$$\frac{a : Int[max = x] \quad b : Int[literal = y]}{(a/b) : T[max = (x/y)]} \text{ (Max and literal)}$$

$$\frac{a : Int[literal = x] \quad b : Int[min = y] \quad x > 0 \quad y > 0}{(a/b) : T[max = (x/y)]} \text{ (Literal and min)}$$

$$\frac{a : Int[literal = x] \quad b : Int[max = y] \quad x > 0 \quad y < 0}{(a/b) : T[min = (x/y)]} \text{ (Literal and max)}$$

$$\frac{a : Int[literal = x] \quad b : Int[min = y] \quad x < 0 \quad y > 0}{(a/b) : T[min = (x/y)]} \text{ (Negative literal and min)}$$

$$\frac{a : Int[literal = x] \quad b : Int[max = y] \quad x < 0 \quad y < 0}{(a/b) : T[max = (x/y)]} \text{ (Negative literal and max)}$$

Multiply And commutative versions.

$$\frac{a : T[literal = x] \quad b : T[literal = y] \quad T \in \{ Int, Float \}}{(a \times b) : T[literal = (x \times y)]} \text{ (Literal)}$$

$$\frac{a : T[literal = 0] \quad b : T \quad T \in \{ Int, Float \}}{(a \times b) : T[literal = 0]} \text{ (Literal zero)}$$

$$\frac{a : T[literal = x] \quad b : T[min = y] \quad T \in \{ Int, Float \} \quad x > 0}{(a \times b) : T[min = (x \times y)]} \text{ (Positive literal and min)}$$

$$\frac{a : T[literal = x] \quad b : T[max = y] \quad T \in \{ Int, Float \} \quad x > 0}{(a \times b) : T[max = (x \times y)]} \text{ (Positive literal and max)}$$

$$\frac{a : T[literal = x] \quad b : T[min = y] \quad T \in \{ Int, Float \} \quad x < 0}{(a \times b) : T[max = (x \times y)]} \text{ (Negative literal and min)}$$

$$\frac{a : T[literal = x] \quad b : T[max = y] \quad T \in \{ Int, Float \} \quad x < 0}{(a \times b) : T[max = (x \times y)]} \text{ (Negative literal and max)}$$

$$\frac{a : T[domain = m] \quad b : T[domain = n] \quad T \in \{ Int, Float \}}{(a \times b) : T[domain = map(product, m \times n)]} \text{ (Domain widens)}$$

Power

$$\frac{a : T[literal = x] \quad b : T[literal = y] \quad T \in \{ Int, Float \}}{(a^b) : T[literal = (x^y)]} \text{ (Literal)}$$

$$\frac{a : T[literal = 0] \quad b : T[literal! = 0] \quad T \in \{ Int, Float \}}{(a^b) : T[literal = 0]} \text{ (Literal zero)}$$

$$\frac{a : T \quad b : T[literal = 0] \quad T \in \{ Int, Float \}}{(a^b) : T[literal = 1]} \text{ (Literal zero)}$$

$$\frac{a : T[literal = x] \quad b : T[min = y] \quad T \in \{ Int, Float \}}{(a^b) : T[min = (x^y)]} \text{ (Literal and min)}$$

$$\frac{a : T[literal = x] \quad b : T[max = y] \quad T \in \{ Int, Float \}}{(a^b) : T[max = (x^y)]} \text{ (Literal and max)}$$

$$\frac{a : T[domain = m] \quad b : T[domain = n] \quad T \in \{\, Int, Float \,\}}{(a^b) : T[domain = map(\lambda(x, y) : x^y, m \times n)]} \quad \text{(Domain widens)}$$

Upto

$$\frac{a : Int[literal = x] \quad b : Int[literal = y]}{(a \,..\, b) : set[min = x, max = y, literal = \{\, x, x + 1, ..., y \,\}, total]} \quad \text{(Upto literal)}$$

$$\frac{a : Int[min = x] \quad b : Int}{(a \,..\, b) : set[min = x]} \quad \text{(Upto literal)}$$

$$\frac{a : Int \quad b : Int[max = x]}{(a \,..\, b) : set[max = x]} \quad \text{(Upto literal)}$$

Tuple

Enumeration

$$\frac{a_1 : T_1, a_2 : T_2, ..., a_n : T_n}{(a_1, a_2, ..., a_n) : tuple[T_1, T_2, ..., T_n, literal = (a_1, a_2, ..., a_n), size = n]} \quad \text{(Literal)}$$

Maplet

$$\frac{a_1 : T_1, a_2 : T_2}{(a_1 \mapsto a_2) : tuple[T_1, T_2, literal = (a_1, a_2), size = 2]} \quad \text{(Literal)}$$

Set Note: the domain trait on sets is not the same as that of primitive types. A primitive value selects one out of the domain, whereas collection types choose from a subset of the domain. For example, let $x : int[domain = \{\, 1, 2, 3 \,\}]$. Then $x \in \{\, 1, 2, 3 \,\}$. However, if $y : set[int, domain = \{\, 1, 2, 3 \,\}]$, then $y \subset \{\, 1, 2, 3 \,\}$ (that is, $y \in \mathbb{P}\{\, 1, 2, 3 \,\}$).

Enumeration

$$\frac{a_1, a_2, ..., a_n : T}{\{\, a_1, a_2, ..., a_n \,\} : set[T, literal = \{\, a_1, a_2, ..., a_n \,\}, size = n]} \quad \text{(Literal)}$$

Clear

$$\frac{s : set[T]}{clear(s) : set[T, literal = \{\ \}, empty]} \quad \text{(Literal)}$$

Is empty

$$\frac{s : set[Empty]}{(s = \{\ \}) : Bool[literal = true]} \quad \text{(Literal)}$$

Add

$$\frac{s : set[T, Size = x] \quad a : T}{(s \cup \{\, a \,\}) : set[T, Size = x + 1]} \quad \text{(Size may increase)}$$

$$\frac{s : set[T, Size = x, Total] \quad a : T}{(s \cup \{\, a \,\}) : set[T, Size = x, Total]} \quad \text{(Total size stays the same)}$$

$$\frac{s : set[T, literal = l_1] \quad a : T[literal = l_2]}{(s \cup \{\, a \,\}) : set[T, literal = (l_1 \cup \{\, l_2 \,\})]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d] \quad a : T[literal = l]}{(s \cup \{\, a \,\}) : set[T, domain = (d \cup \{\, l \,\})]} \quad \text{(Domain with literal)}$$

$$\frac{s : set[T, domain = d] \quad a : T[domain = l]}{(s \cup \{\, a \,\}) : set[T, literal = (d \cup l)]} \quad \text{(Domain with domain)}$$

$$\frac{s : set[T, empty] \quad a : T}{(s \cup \{\, a \,\}) : set[T]} \quad \text{(Empty)}$$

Remove

$$\frac{s : set[T, size = n] \quad a : T}{(s \setminus \{\, a \,\}) : set[T, size = n]} \quad \text{(Size does not guarantee a decrease)}$$

$$\frac{s : set[T, size = x, total] \quad a : T}{(s \setminus \{\, a \,\}) : set[T, size = x - 1]} \quad \text{(Removes Total guarantee)}$$

$$\frac{s : set[T, literal = l_1] \quad a : T[literal = l_2]}{(s \setminus \{\, a \,\}) : set[T, literal = (l_1 \setminus \{\, l_2 \,\})]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d] \quad a : T[literal = l]}{(s \setminus \{\, a \,\}) : set[T, domain = (d \setminus \{\, l \,\})]} \quad \text{(Domain with literal)}$$

$$\frac{s : set[T, empty] \quad a : T}{(s \setminus \{\, a \,\}) : set[T, empty]} \quad \text{(Empty)}$$

In Not in is similar.

$$\frac{s : set[T, literal = l_1] \quad a : T[literal = l_2]}{(a \in s) : Bool[literal = (l_2 \in l_1)]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d, total] \quad a : T[literal = l]}{(a \in s) : Bool[literal = (l \in d)]} \quad \text{(Total domain with literal)}$$

$$\frac{s : set[T, domain = d_1, total] \quad a : T[domain = d_2]}{(a \in s) : Bool[literal = (d_2 \subseteq d_1)]} \quad \text{(Total domain with domain)}$$

$$\frac{s : set[T, empty] \quad a : T}{(a \in s) : Bool[literal = false]} \quad \text{(Empty)}$$

Cardinality

$$\frac{s : set[T, size = n]}{card(s) : Int[max = n]} \quad \text{(Size overestimates)}$$

$$\frac{s : set[T, literal = l]}{card(s) : Int[literal = card(l)]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d]}{card(s) : Int[max = card(d)]} \quad \text{(Domain)}$$

$$\frac{s : set[T, empty]}{card(s) : Int[literal = 0]} \quad \text{(Empty)}$$

Powerset

$$\frac{s : set[T, size = n, total]}{powerset(s) : set[set[T], size = 2^n, total]} \quad \text{(Total size)}$$

$$\frac{s : set[T, literal = l]}{powerset(s) : set[set[T], literal = \mathbb{P}(l)]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d]}{powerset(s) : set[set[T], domain = \mathbb{P}(d)]} \quad \text{(Domain)}$$

Choice

$$\frac{s : set[T, empty]}{choice(s) : T[undefined]} \quad \text{(Empty choice fails (statically))}$$

$$\frac{s : set[T, literal = \{\, l \,\}]}{choice(s) : T[literal = l]} \quad \text{(Single literal)}$$

$$\frac{s : set[T, domain = d]}{choice(s) : T[domain = l]} \quad \text{(Domain)}$$

Min

$$\frac{s : set[T, min = m]}{min(s) : T[literal = m]} \quad \text{(Min)}$$

$$\frac{s : set[T, literal = \{\, l \,\}]}{min(s) : T[literal = l]} \quad \text{(Single literal)}$$

$$\frac{s : set[T, literal = l]}{min(s) : T[domain = l]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d]}{min(s) : T[domain = d]} \quad \text{(Domain)}$$

Max

$$\frac{s : set[T, max = m]}{max(s) : T[literal = m]} \quad \text{(Max)}$$

$$\frac{s : set[T, literal = \{\, l \,\}]}{max(s) : T[literal = l]} \quad \text{(Single literal)}$$

$$\frac{s : set[T, literal = l]}{max(s) : T[domain = l]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d]}{max(s) : T[domain = d]} \quad \text{(Domain)}$$

Sum

$$\frac{s : set[T, max = y, size = n]}{sum(s) : T[max = y \times n]} \quad \text{(Sum bounds)}$$

$$\frac{s : set[T, literal = l]}{sum(s) : T[literal = sum(l)]} \quad \text{(Literal)}$$

Product

$$\frac{s : set[T, max = y, size = n]}{product(s) : T[max = y^n]} \quad \text{(Product bounds)}$$

$$\frac{s : set[T, literal = l]}{product(s) : T[literal = product(l)]} \quad \text{(Literal)}$$

Union

$$\frac{s : set[T, literal = l_1] \quad t : set[T, literal = l_2]}{(s \cup t) : set[T, literal = (l_1 \cup l_2)]} \quad \text{(Literal)}$$

$$\frac{s : set[T, domain = d_1] \quad t : set[T, domain = d_2]}{(s \cup t) : set[T, domain = (d_1 \cup d_2)]} \quad \text{(Domain)}$$

$$\frac{s : set[T, domain = d] \quad t : set[T, literal = l]}{(s \cup t) : set[T, domain = (d \cup l)]} \quad \text{(Domain and Literal)}$$

$$\frac{s : set[T, min = m_1] \quad t : set[T, min = m_2] \quad m_1 < m_2}{(s \cup t) : set[T, min = m_1]} \quad \text{(Lowest Min)}$$

$$\frac{s : set[T, max = m_1] \quad t : set[T, max = m_2] \quad m_1 < m_2}{(s \cup t) : set[T, max = m_2]} \quad \text{(Highest Max)}$$

$$\frac{s : set[T, empty] \quad t : set[T, traits]}{(s \cup t) : set[T, traits]} \quad \text{(Union with Empty keeps Traits)}$$

$$\frac{s : set[T, size = x] \quad t : set[T, size = y]}{(s \cup t) : set[T, size = (x + y)]} \quad \text{(Size overestimates)}$$

$$\frac{s : set[T, domain = d_1] \quad t : set[T, domain = d_2, total] \quad d_1 \subseteq d_2}{(s \cup t) : set[T, domain = d_2, total]}$$ (Total Domain)

Intersection

$$\frac{s : set[T, literal = l_1] \quad t : set[T, literal = l_2]}{(s \cap t) : set[T, literal = (l_1 \cap l_2)]}$$ (Literal)

$$\frac{s : set[T, domain = d_1] \quad t : set[T, domain = d_2]}{(s \cap t) : set[T, domain = (d_1 \cap d_2)]}$$ (Domain)

$$\frac{s : set[T, domain = d] \quad t : set[T, literal = l]}{(s \cap t) : set[T, domain = (d \cap l)]}$$ (Domain and Literal)

$$\frac{s : set[T, min = m_1] \quad t : set[T, min = m_2] \quad m_1 < m_2}{(s \cap t) : set[T, min = m_2]}$$ (Highest Min)

$$\frac{s : set[T, max = m_1] \quad t : set[T, max = m_2] \quad m_1 < m_2}{(s \cap t) : set[T, max = m_1]}$$ (Lowest Max)

$$\frac{s : set[T, empty] \quad t : set[T]}{(s \cap t) : set[T, empty]}$$ (Intersection with Empty is Empty)

$$\frac{s : set[T, size = x] \quad t : set[T, size = y]}{(s \cap t) : set[T, size = max(x, y)]}$$ (Size does not increase)

$$\frac{s : set[T, domain = d_1] \quad t : set[T, domain = d_2, total] \quad d_1 \subseteq d_2}{(s \cup t) : set[T, domain = d_1]}$$ (Total Domain)

Difference

$$\frac{s : set[T, literal = l_1] \quad t : set[T, literal = l_2]}{(s \setminus t) : set[T, literal = (l_1 \setminus l_2)]}$$ (Literal)

$$\frac{s : set[T, literal = l] \quad t : set[T]}{(s \setminus t) : set[T, domain = l]}$$ (Literal)

$$\frac{s : set[T, domain = d] \quad t : set[T]}{(s \setminus t) : set[T, domain = d]}$$ (Domain)

$$\frac{s : set[T, domain = d] \quad t : set[T, literal = l]}{(s \setminus t) : set[T, domain = (d \setminus l)]}$$ (Domain and Literal)

$$\frac{s : set[T, traits] \quad t : set[T, empty]}{(s \setminus t) : set[T, traits]}$$ (Difference with Empty keeps traits)

$$\frac{s : set[T, empty] \quad t : set[T]}{(s \setminus t) : set[T, empty]}$$ (Empty difference)

$$\frac{s : set[T, size = x] \quad t : set[T]}{(s \setminus t) : set[T, size = x]}$$ (Size does not increase)

Cartesian product

$$\frac{s : set[T, literal = l_1] \quad t : set[V, literal = l_2]}{(s \times t) : relation[T, V, literal = l_1 \times l_2]} \text{ (Literal)}$$

$$\frac{s : set[T, domain = d_1] \quad t : set[V, domain = d_2]}{(s \times t) : relation[T, V, domain = d_1 \times d_2]} \text{ (Domain)}$$

$$\frac{s : set[T, domain = d] \quad t : set[V, literal = l]}{(s \times t) : relation[T, V, domain = d \times l]} \text{ (Domain and Literal)}$$

$$\frac{s : set[T, literal = l] \quad t : set[V, domain = d]}{(s \times t) : relation[T, V, literal = l \times d]} \text{ (Literal and Domain)}$$

$$\frac{s : set[T, domain = d_1, total] \quad t : set[V, domain = d_2, total]}{(s \times t) : relation[T, V, domain = d_1 \times d_2, total]} \text{ (Total)}$$

$$\frac{s : set[T, size = x] \quad t : set[V, size = y]}{(s \times t) : relation[T, V, size = x \times y]} \text{ (Size)}$$

$$\frac{s : set[T, empty] \quad t : set[V]}{(s \times t) : relation[T, V, empty]} \text{ (Empty)}$$

$$\frac{s : set[T] \quad t : set[V, empty]}{(s \times t) : relation[T, V, empty]} \text{ (Empty)}$$

Subset And negation/superset variant.

$$\frac{s : set[T, literal = l_1] \quad t : set[T, literal = l_2]}{(s \subset t) : Bool[literal = l_1 \subset l_2]} \text{ (Literal)}$$

$$\frac{s : set[T, domain = d_1] \quad t : set[T, domain = d_2] \quad d_1 \cap d_2 = \varnothing}{(s \subset t) : Bool[literal = false]} \text{ (Domains do not overlap)}$$

$$\frac{s : set[T, min = m_1] \quad t : set[T, max = m_2] \quad m_1 > m_2}{(s \subset t) : Bool[literal = false]} \text{ (Min above Max)}$$

$$\frac{s : set[T, max = m_1] \quad t : set[T, min = m_2] \quad m_1 < m_2}{(s \subset t) : Bool[literal = false]} \text{ (Max below Min)}$$

$$\frac{s : set[T, empty] \quad t : set[T, \neg empty]}{(s \subset t) : Bool[literal = true]} \text{ (Empty is subset of nonempty)}$$

Subset equals And negation/superset variant.

$$\frac{s : set[T, literal = l_1] \quad t : set[T, literal = l_2]}{(s \subseteq t) : Bool[literal = l_1 \subseteq l_2]} \text{ (Literal)}$$

$$\frac{s : set[T, domain = d_1, \neg empty] \quad t : set[T, domain = d_2] \quad d_1 \cap d_2 = \varnothing}{(s \subset t) : Bool[literal = false]} \text{ (Nonempty Domains do not overlap)}$$

$$\frac{s : set[T, min = m_1, \neg empty] \quad t : set[T, max = m_2] \quad m_1 > m_2}{(s \subset t) : Bool[literal = false]} \text{ (Nonempty Min above Max)}$$

$$\frac{s : set[T, max = m_1, \neg empty] \quad t : set[T, min = m_2] \quad m_1 < m_2}{(s \subset t) : Bool[literal = false]}$$ (Nonempty Max below Min)

$$\frac{s : set[T, empty] \quad t : set[T]}{(s \subset t) : Bool[literal = true]}$$ (Empty is subset-equals of all)

Relation refinements Relations have 4 tightly-coupled flag-like trait refinements that influence one another and ultimately make up relational subtypes (like totality, surjectivity, injectivity, etc.). For conciseness, we will express the following operation-trait interactions as a boolean mask representing [with total (t), with total on range (t_r), with one-to-manyness ($1-$), with many-to-oneness ($1-_r$)]. Subtyping is done conservatively: we keep subtype properties only when they are guaranteed. See Section 5.1.3 for more on relational subtyping.

$$\frac{s : set[T_1] \quad r : relation[T_1, T_2, [true, x, y, z]]}{(s \lhd r) : relation[T_1, T_2, [false, x, y, z]]}$$ (Relational Subtype - Domain Restriction)

$$\frac{s : set[T_1] \quad r : relation[T_1, T_2, [true, x, y, z]]}{(s \lhd\!\!\!- r) : relation[T_1, T_2, [false, x, y, z]]}$$ (Relational Subtype - Domain Subtraction)

$$\frac{s : set[T_2] \quad r : relation[T_1, T_2, [x, true, y, z]]}{(r \rhd s) : relation[T_1, T_2, [x, false, y, z]]}$$ (Relational Subtype - Range Restriction)

$$\frac{s : set[T_2] \quad r : relation[T_1, T_2, [x, true, y, z]]}{(r \rhd\!\!\!- s) : relation[T_1, T_2, [x, false, y, z]]}$$ (Relational Subtype - Range Subtraction)

$$\frac{r : relation[T_1, T_2, [w, x, y, z]]}{r^{-1} : relation[T_1, T_2, [x, w, z, y]]}$$ (Relational Subtype - Inverse)

$$\frac{r : relation[T_1, T_2, [a, b, c, d]] \quad t : relation[T_1, T_2, [w, x, y, z]]}{(r \oplus t) : relation[T_1, T_2, [w, x, c \wedge y, d \wedge z]]}$$ (Relational Subtype - Overriding)

$$\frac{r : relation[T_1, T_2, [a, b, c, d]] \quad t : relation[T_2, T_3, [w, x, y, z]]}{(r \circ t) : relation[T_1, T_3, [a, b \wedge x, c \wedge y, d \wedge z]]}$$ (Relational Subtype - Composition)

Relation This section covers additional traits aside from the above subtype mask section.

Inverse

$$\frac{r : relation[T_1, T_2, literal = l]}{r^{-1} : relation[T_2, T_1, literal = l^{-1}]}$$ (Literal)

$$\frac{r : relation[T_1, T_2, relationalDomain = d, relationalRange = r]}{r^{-1} : relation[T_2, T_1, relationalDomain = r, relationalRange = d]}$$ (Relational Domain)

$$\frac{r : relation[T_1, T_2, size = n]}{r^{-1} : relation[T_2, T_1, size = n]}$$ (Size)

$$\frac{r : relation[T_1, T_2, empty]}{r^{-1} : relation[T_2, T_1, empty]}$$ (Empty)

$$\frac{r : relation[T_1, T_2, relationalDomainMin = m]}{r^{-1} : relation[T_2, T_1, relationalDomainMax = m]}$$ (Relational Domain min swap)

$$\frac{r : relation[T_1, T_2, relationalDomainMax = m]}{r^{-1} : relation[T_2, T_1, relationalDomainMin = m]} \quad \text{(Relational Domain max swap)}$$

$$\frac{r : relation[T_1, T_2, relationalRangeMax = m]}{r^{-1} : relation[T_2, T_1, relationalRangeMin = m]} \quad \text{(Relational Range min swap)}$$

$$\frac{r : relation[T_1, T_2, relationalRangeMax = m]}{r^{-1} : relation[T_2, T_1, relationalRangeMin = m]} \quad \text{(Relational Range max swap)}$$

Composition

$$\frac{s : relation[T, V, literal = l_1] \quad t : relation[V, U, literal = l_2]}{s \circ t : relation[T, U, literal = l_1 \circ l_2]} \quad \text{(Literal)}$$

$$\frac{s : relation[T, V, relationalDomain = d] \quad t : relation[V, U, relationalRange = r]}{s \circ t : relation[T, U, relationalDomain = d, relationalRange = r]} \quad \text{(Domain)}$$

$$\frac{s : relation[T, V, empty] \quad t : relation[V, U]}{s \circ t : relation[T, U, empty]} \quad \text{(Empty)}$$

$$\frac{s : relation[T, V] \quad t : relation[V, U, empty]}{s \circ t : relation[T, U, empty]} \quad \text{(Empty)}$$

$$\frac{s : relation[T, V, size = n_1] \quad t : relation[V, U, size = n_2]}{s \circ t : relation[T, U, size = max(n_1, n_2)]} \quad \text{(Size)}$$

$$\frac{s : relation[T, V, extremum = m] \quad t : relation[V, U] \quad extremum \in \{ relationalDomainMin, relationalDomainMin \}}{s \circ t : relation[T, U, extremum = m]}$$

(Relational domain min/max persists (1st arg))

$$\frac{s : relation[T, V] \quad t : relation[V, U, extremum = m] \quad extremum \in \{ relationalRangeMin, relationalRangeMin \}}{s \circ t : relation[T, U, extremum = m]}$$

(Relational range min/max persists (2nd arg))

Image

$$\frac{r : relation[T, V, literal = l_1] \quad s : set[T, literal = l_2]}{r[s] : set[V, literal = l_1[l_2]]} \quad \text{(Literal)}$$

$$\frac{r : relation[T, V, relationalDomain = d] \quad s : set[T, literal = l] \quad l \not\subseteq d}{r[s] : set[V, empty]} \quad \text{(Domain does not contain literal)}$$

$$\frac{r : relation[T, V, relationalDomain = d_1] \quad s : set[T, domain = d_2] \quad d_2 \not\subseteq d_1}{r[s] : set[V, empty]} \quad \text{(Domain does not contain domain)}$$

$$\frac{r : relation[T, V, relationalDomain = d_1, TotalOnDomain] \quad s : set[T, domain = d_2, size > 0] \quad d_2 \subseteq d_1}{r[s] : set[V, size > 0]}$$

(Total nonempty domains)

$$\frac{r : relation[T, V, relationalDomainMax = m_1] \quad s : set[T, min = m_2] \quad m_1 < m_2}{r[s] : set[V, empty]} \quad \text{(Arg min above relational max)}$$

$$\frac{r : relation[T, V, relationalDomainMin = m_1] \quad s : set[T, max = m_2] \quad m_2 < m_1}{r[s] : set[V, empty]}$$ (Arg max below relational min)

$$\frac{r : relation[T, V] \quad s : set[T, empty]}{r[s] : set[V, empty]}$$ (Empty arg)

$$\frac{r : relation[T, V, empty] \quad s : set[T]}{r[s] : set[V, empty]}$$ (Empty relation)

$$\frac{r : relation[T, V, ManyToOne] \quad s : set[T, size = n]}{r[s] : set[V, size = n]}$$ (Size does not increase with many-to-one)

Overriding

$$\frac{r : relation[T_1, T_2, literal = l_1] \quad t : relation[T_1, T_2, literal = l_2]}{(r \oplus t) : relation[T_1, T_2, literal = l_1 \oplus l_2]}$$ (Literal)

$$\frac{r : relation[T_1, T_2, relationalDomain = d_1] \quad t : relation[T_1, T_2, relationalDomain = d_2]}{(r \oplus t) : relation[T_1, T_2, relationalDomain = d_1 \cup d_2]}$$ (Relational Domain)

$$\frac{r : relation[T_1, T_2, relationalRange = r_1] \quad t : relation[T_1, T_2, relationalRange = r_2]}{(r \oplus t) : relation[T_1, T_2, relationalRange = r_1 \cup r_2]}$$ (Relational Range)

$$\frac{r : relation[T_1, T_2, size = n_1] \quad t : relation[T_1, T_2, size = n_2]}{(r \oplus t) : relation[T_1, T_2, size = n_1 + n_2]}$$ (Size overestimates)

$$\frac{r : relation[T_1, T_2, empty] \quad t : relation[T_1, T_2, traits]}{(r \oplus t) : relation[T_1, T_2, traits]}$$ (Empty overriding keeps traits (1))

$$\frac{r : relation[T_1, T_2, traits] \quad t : relation[T_1, T_2, empty]}{(r \oplus t) : relation[T_1, T_2, traits]}$$ (Empty overriding keeps traits (2))

$$\frac{r : relation[T_1, T_2, relationalDomainMin = m_1] \quad t : relation[T_1, T_2, relationalDomainMin = m_2]}{(r \oplus t) : relation[T_1, T_2, relationalDomainMin = min(m_1, m_2)]}$$ (Relational domain min)

$$\frac{r : relation[T_1, T_2, relationalDomainMax = m_1] \quad t : relation[T_1, T_2, relationalDomainMax = m_2]}{(r \oplus t) : relation[T_1, T_2, relationalDomainMax = max(m_1, m_2)]}$$ (Relational domain max)

$$\frac{r : relation[T_1, T_2, relationalRangeMin = m_1] \quad t : relation[T_1, T_2, relationalRangeMin = m_2]}{(r \oplus t) : relation[T_1, T_2, relationalRangeMin = min(m_1, m_2)]}$$ (Relational range min)

$$\frac{r : relation[T_1, T_2, relationalRangeMax = m_1] \quad t : relation[T_1, T_2, relationalRangeMax = m_2]}{(r \oplus t) : relation[T_1, T_2, relationalRangeMax = max(m_1, m_2)]}$$ (Relational range max)

Domain

$$\frac{r : relation[T, V, literal = l]}{dom(r) : set[T, literal = dom(l)]}$$ (Literal)

$$\frac{r : relation[T, V, relationalDomain = d]}{dom(r) : set[T, domain = d]} \quad \text{(Relational Domain)}$$

$$\frac{r : relation[T, V, TotalOnDomain]}{dom(r) : set[T, Total]} \quad \text{(Total domain)}$$

$$\frac{r : relation[T, V, OneToMany, ManyToOne, size = n]}{dom(r) : set[T, size = n]} \quad \text{(One-to-one maintains size)}$$

$$\frac{r : relation[T, V, empty]}{dom(r) : set[T, empty]} \quad \text{(Empty)}$$

Range

$$\frac{r : relation[T, V, literal = l]}{ran(r) : set[V, literal = ran(l)]} \quad \text{(Literal)}$$

$$\frac{r : relation[T, V, relationalRange = r]}{ran(r) : set[V, range = r]} \quad \text{(Relational Range)}$$

$$\frac{r : relation[T, V, TotalOnRange]}{ran(r) : set[V, Total]} \quad \text{(Total range)}$$

$$\frac{r : relation[T, V, OneToMany, ManyToOne, size = n]}{ran(r) : set[V, size = n]} \quad \text{(One-to-one maintains size)}$$

$$\frac{r : relation[T, V, empty]}{ran(r) : set[V, empty]} \quad \text{(Empty)}$$

Domain restriction

$$\frac{s : set[T_1, literal = l_1] \qquad r : relation[T_1, T_2, literal = l_2]}{(s \lhd r) : relation[T_1, T_2, literal = l_1 \lhd l_2]} \quad \text{(Literal)}$$

$$\frac{s : set[T_1, domain = d] \qquad r : relation[T_1, T_2]}{(s \lhd r) : relation[T_1, T_2, domain = d]} \quad \text{(Domain)}$$

Domain subtraction

$$\frac{s : set[T_1, literal = l_1] \qquad r : relation[T_1, T_2, literal = l_2]}{(s \lhd\!\!\!- r) : relation[T_1, T_2, literal = l_1 \lhd\!\!\!- l_2]} \quad \text{(Literal)}$$

$$\frac{s : set[T_1, domain = d_1] \qquad r : relation[T_1, T_2, relationalDomain = d_2]}{(s \lhd\!\!\!- r) : relation[T_1, T_2, relationalDomain = d_2 \setminus d_1]} \quad \text{(Domain)}$$

Range restriction

$$\frac{s : set[T_2, literal = l_1] \qquad r : relation[T_1, T_2, literal = l_2]}{(r \rhd s) : relation[T_1, T_2, literal = l_2 \rhd l_1]} \quad \text{(Literal)}$$

$$\frac{s : set[T_2, domain = d] \qquad r : relation[T_1, T_2]}{(r \rhd s) : relation[T_1, T_2, relationalRange = d]} \quad \text{(Domain)}$$

Range subtraction

$$\frac{s : set[T_2, literal = l_1] \quad r : relation[T_1, T_2, literal = l_2]}{(r \rhd s) : relation[T_1, T_2, literal = l_2 \rhd l_1]} \text{(Literal)}$$

$$\frac{s : set[T_2, domain = d] \quad r : relation[T_1, T_2, relationalRange = r]}{(r \rhd s) : relation[T_1, T_2, relationalRange = r \setminus d]} \text{(Domain)}$$

Bag Image

$$\frac{r : relation[T_1, T_2, literal = l_1] \quad b : bag[T_1, literal = l_2]}{(r[b]) : bag[T_2, literal = l_1[l_2]]} \text{(Literal)}$$

Bag

Bag union

$$\frac{s : bag[T, literal = l_1] \quad t : bag[T, literal = l_2]}{(s \cup t) : bag[T, literal = (l_1 \cup l_2)]} \text{(Literal)}$$

$$\frac{s : bag[T, domain = d_1] \quad t : bag[T, domain = d_2]}{(s \cup t) : bag[T, domain = (d_1 \cup d_2)]} \text{(Domain)}$$

$$\frac{s : bag[T, size = n_1] \quad t : bag[T, size = n_2]}{(s \cup t) : bag[T, size = n_1 + n_2]} \text{(Size)}$$

Bag intersection

$$\frac{s : bag[T, literal = l_1] \quad t : bag[T, literal = l_2]}{(s \cap t) : bag[T, literal = (l_1 \cap l_2)]} \text{(Literal)}$$

$$\frac{s : bag[T, domain = d_1] \quad t : bag[T, domain = d_2]}{(s \cap t) : bag[T, domain = (d_1 \cap d_2)]} \text{(Domain)}$$

$$\frac{s : bag[T, size = n_1] \quad t : bag[T, size = n_2]}{(s \cap t) : bag[T, size = min(n_1, n_2)]} \text{(Size)}$$

Bag add

$$\frac{s : bag[T, literal = l_1] \quad t : bag[T, literal = l_2]}{(s + t) : bag[T, literal = (l_1 + l_2)]} \text{(Literal)}$$

$$\frac{s : bag[T, domain = d_1] \quad t : bag[T, domain = d_2]}{(s + t) : bag[T, domain = (d_1 \cup d_2)]} \text{(Domain)}$$

$$\frac{s : bag[T, size = n_1] \quad t : bag[T, size = n_2]}{(s + t) : bag[T, size = n_1 + n_2]} \text{(Size)}$$

Bag difference

$$\frac{s : bag[T, literal = l_1] \quad t : bag[T, literal = l_2]}{(s - t) : bag[T, literal = (l_1 - l_2)]} \text{ (Literal)}$$

$$\frac{s : bag[T, domain = d] \quad t : bag[T]}{(s - t) : bag[T, domain = d]} \text{ (Domain)}$$

$$\frac{s : bag[T, size = n] \quad t : bag[T]}{(s - t) : bag[T, size = n]} \text{ (Size)}$$

Size

$$\frac{b : bag[T, literal = l]}{size(b) : Int[literal = size(l)]} \text{ (Literal)}$$

$$\frac{b : bag[T, domain = d]}{size(b) : Int[max = size(d)]} \text{ (Domain)}$$

$$\frac{b : bag[T, size = n]}{size(b) : Int[min = n]} \text{ (Size)}$$

Sequence

Concat

$$\frac{s : sequence[T, literal = l_1] \quad t : sequence[T, literal = l_2]}{(s +\!\!+ t) : sequence[T, literal = l_1 +\!\!+ l_2]} \text{ (Literal)}$$

$$\frac{s : sequence[T, size = x] \quad t : sequence[T, size = y]}{(s +\!\!+ t) : sequence[T, size = x + y]} \text{ (Size)}$$

Quantification Body

Generator

$$\frac{s : set[T, traits] \quad i : tupleIdentifier}{(i \in s) : Generator[T, traits]} \text{ (Generator keeps traits)}$$

$$\frac{s : set[T, traits, total] \quad i : tupleIdentifier \quad p : Bool}{(i \in s \cdot p) : Generator[T, traits]} \text{ (Generator keeps traits)}$$

$$\frac{g_i : Generator[T, literal = l_i]}{(g_1, ..., g_n) : Generator[T, literal = l_1 \times ... \times l_n]} \text{ (Literal)}$$

$$\frac{g_i : Generator[T, domain = d_i]}{(g_1, ..., g_n) : Generator[T, domain = d_1 \times ... \times d_n]} \text{ (Domain)}$$

Quantifier

$$\frac{g : Generator[T, literal = l] \quad e : T \to E}{(g \mid e) : Quantifier[E, literal = e(g)]} \text{(Literal)}$$

$$\frac{q_i : Quantifier[T, literal = l_i]}{(g_1 `...` g_n) : Generator[T, literal = l_1 \cup ... \cup l_n]} \text{(Literal)}$$

$$\frac{q_i : Quantifier[T, domain = d_i]}{(q_1 `...` q_n) : Generator[T, domain = d_1 \cup ... \cup d_n]} \text{(Domain)}$$

Comprehension

$$\frac{q : Quantifier[T, literal = l]}{\{\, q \,\} : set[T, literal = l]} \text{(Literal)}$$

$$\frac{q : Quantifier[T, domain = d]}{\{\, q \,\} : set[T, domain = d]} \text{(Domain)}$$

Map

$$\frac{s : set[T, literal = l] \quad f : T \to E}{map(f, q) : set[E, literal = map(f, l)]} \text{(Literal)}$$

Fold

$$\frac{s : set[T, literal = l] \quad f : T \times T \to T \quad i : T[literal = l_i]}{fold(f, i, s) : set[T, literal = fold(f, i, l)]} \text{(Literal)}$$

2.5 Operational Semantics

Let σ represent the symbol table (data store) for assigned variables, defined types, and procedure definitions. Further, let $\perp$ represent an error state; any operations that do not explicitly handle the error state are expected to propagate the error. The following sections define small-step semantics for Simile in the form of inference statements from $\langle$Simile code, $\sigma\rangle \to \langle$Derived simile code, $\sigma'\rangle$. Generally, notation in the hypothesis section of each statement will correspond to set theory; notation in the inference corresponds to Simile code.

For brevity, declaring a variable in the hypothesis implicitly means that value is available within σ prior to execution. As an example:

$$v_1 : T_1, v_2 : T_2, ... \equiv v_1 \in \sigma \wedge type(v_1) = T_1, v_2 \in \sigma \wedge type(v_2) = T_2, ...$$

When σ does not change between steps, it is also excluded. For example:

$$\texttt{v_1 + v_2} \equiv \langle \texttt{v_1 + v_2}, \sigma \rangle$$

Values of Types The following are models of value sets in Simile. Although we try to follow set theory where possible, some types opt for more conventional programming styles for ease of use and relevance to programming. Let value v be defined as any of the following:

v ::= a | b | i | f | q | t | s | r | B | S | R

None

a ::= ‘`none`’

Bool

b ::= ‘`true`’ | ‘`false`’

Int

i ::= [-][0-9]+

where $i \in -\infty \,..\, \infty$

Float Let i_N be an int constrained such that $-2^{N-1} + 1 \leq i_N \leq 2^{N-1}$. Then,

f ::= [-][0-9]*‘.’[0-9]+

64-bit floats following IEEE-754 with variables $e_{bits} = 11, m_{bits} = 52, e = 2^{e_{bits}} - 1023, m = 2 \times 2^{m_{bits}}$ where $-2^e \times m \leq f \leq 2^e \times m$. Effectively, floats follow $-1.79 \times 10^{308} \leq f \leq 1.79 \times 10^{308}$.

String

q ::= ‘`"`’ [^‘`"`’]* ‘`"`’

with additional semantics like:

q’ ::= ‘`{}`’ | ‘`{`’i ‘↦’ q {‘`,`’ i ‘↦’ q }‘`}`’

where $min(dom(q')) = 0$ and $max(dom(q')) = |q'|$ (ie. it is a sequence of characters).

Tuple

t ::= ‘`()`’ | ‘`(`’ value ‘`,)`’ | ‘`(`’ value ‘`,`’ value {‘`,`’ value}‘`)`’

Set

s ::= ‘`{}`’ | ‘`{`’ value {‘`,`’ value}‘`}`’

Relation

r ::= ‘`{}`’ | ‘`{`’ value ‘↦’ value {‘`,`’ value ‘↦’ value}‘`}`’

Further, $relation[T_1, T_2] = set[(T_1, T_2)]$ and carry all semantics associated with that type.

Bag

B ::= ‘`{}`’ | ‘`{`’ value ‘↦’ i {‘`,`’ value ‘↦’ i}‘`}`’

where $\forall i \in ran(B) | i \geq 0$. Further, $bag[T] = relation[T, int]$ and carry all semantics associated with that type.

Sequence

S ::= '{}' | '{' i '↦' value {',' i '↦' value}'}'

where $min(dom(S)) = 0$ and $max(dom(S)) = |S|$, Further, $sequence[T] = relation[int, T]$ and carry all semantics associated with that type.

Record

R ::= constructor '()' | constructor '('value {',' value}')'

where the constructor identifier is the same as the type identifier. Records act like a named, ordered tuple; effectively a $relation[str, T]$ where T may be any combination of types. Let record R' be defined with fields $f_1, f_2, ...$ Then the view $R' : relation[str, T] = \{ f_1 \mapsto v_1, f_2 \mapsto v_2, ... \}$ can be used. The type is inherently ordered, so there is a another relational view: $R_{type} : relation[int, type[T]] = \{ 0 \mapsto v_{type_1}, 1 \mapsto v_{type_2}, ... \}$. image on the type

Base Environment

$$\frac{}{\{ int, float, string, bool, set, sequence, bag, relation, generic, type, enum, tuple, procedure, record, trait, \mathbb{Z}, \mathbb{N}, \mathbb{N}_1 \} \subseteq \sigma}$$ (Built-in types)

$$\frac{}{\{ immutable, undefined, orderable, expr_procedure, one_to_many, many_to_one, one_to_one \} \subseteq \sigma}$$ (Built-in traits 1)

$$\frac{}{\{ total_on_domain, total_on_range, iterable, unique, empty, total, literal, minimum, maximum, domain, size, bound \} \subseteq \sigma}$$ (Built-in traits 2)

Base Operations

Cast The below table shows allowed type casts (where the source type is on the left and the target type is on top), and judgements define how casting works. Casts that are marked with 7 result in an error value of $\perp$.

	none	bool	int	float	str	tuple	set	relation	bag	sequence	record
none	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗
bool	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗	✗
int	✗	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗
float	✗	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗
str	✗	✓	✓	✓	✓	✓	✗	✗	✗	✓	✗
tuple	✗	✗	✗	✗	✓	✓	✓	✗	✓	✓	✓
set	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✗
relation	✗	✗	✗	✗	✓	✓	✓	✓	✗	✓	✗
bag	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✗
sequence	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✗
record	✗	✗	✗	✗	✓	✓	✓	✓	✗	✓	✓

$$\frac{a : T \quad T : type}{\texttt{cast(a,T)} \to \texttt{a}}$$ (Self typecast)

$$\frac{}{\texttt{cast(none, str)} \to \texttt{``none"}}$$ (None-to-string typecast)

$$\frac{b : bool \quad i : int \quad b \implies (i = 1) \quad \neg b \implies (i = 0)}{\texttt{cast(b, int)} \to \texttt{b}}$$ (Bool-to-int typecast)

$$\frac{}{\texttt{cast(true, str)} \to \texttt{``true''}}$$ (Bool-to-string typecast (true))

$$\frac{}{\texttt{cast(false, str)} \to \texttt{``false''}}$$ (Bool-to-string typecast (false))

$$\frac{i : int \quad f : float = i}{\texttt{cast(i, float)} \to \texttt{f}}$$ (Int-to-float typecast)

$$\frac{i : int \quad q : str = \text{“}i\text{”}}{\texttt{cast(i, str)} \to \texttt{q}}$$ (Int-to-string typecast)

$$\frac{}{\texttt{cast(0, bool)} \to \texttt{false}}$$ (Int-to-bool typecast (false))

$$\frac{i : int \quad i \neq 0}{\texttt{cast(i, bool)} \to \texttt{true}}$$ (Int-to-bool typecast (true))

$$\frac{f : float \quad i : int = \lfloor f \rfloor}{\texttt{cast(f, int)} \to \texttt{i}}$$ (Float-to-int typecast)

$$\frac{f, f_4 : float \quad f > 0 \implies (f_4 = \frac{\lfloor f \times 10^4 \rfloor}{10^4}) \quad f < 0 \implies (f_4 = \frac{\lceil f \times 10^4 \rceil}{10^4}) \quad q : str = \text{“}f_4\text{”}}{\texttt{cast(f, str)} \to \texttt{q}}$$ (Float-to-string typecast)

$$\frac{}{\texttt{cast(0.0, bool)} \to \texttt{false}}$$ (Float-to-bool typecast (false))

$$\frac{f : float \quad f \neq 0.0}{\texttt{cast(f, bool)} \to \texttt{true}}$$ (Float-to-bool typecast (true))

$$\frac{q : str}{\texttt{cast(q, int)} \to \texttt{cast(cast(q, float), int)}}$$ (String-to-int typecast)

$$\frac{q : str \quad q \in [-][0-9] * \backslash .[0-9]+ \quad f : float = q}{\texttt{cast(q, float)} \to \texttt{f}}$$ (String-to-float typecast)

$$\frac{}{\texttt{cast("false", bool)} \to \texttt{false}}$$ (String-to-bool typecast (false))

$$\frac{}{\texttt{cast("true", bool)} \to \texttt{true}}$$ (String-to-bool typecast (true))

$$\frac{q : str \quad S : sequence[str] = [\, c \in q \mid c \,]}{\texttt{cast(q, sequence)} \to \texttt{S}}$$ (String-to-sequence typecast)

$$\frac{q : str}{\texttt{cast(q, tuple)} \to \texttt{cast(cast(q, sequence), tuple)}}$$ (String-to-tuple typecast)

$$\frac{t : (T_1, T_2, ...) \quad t_q = (\, v \in t \mid cast(v, str)\,) \quad q : \text{“}t_q\text{”}}{\texttt{cast(t, str)} \to \texttt{q}}$$ (Tuple-to-string typecast)

$$\frac{t : (T, ...) \quad s : set[T] = \{\, v \in t \mid v \,\}}{\texttt{cast(t, set)} \to \texttt{s}}$$ (Tuple-to-set typecast)

$$\frac{t : (T, ...) \quad S : sequence[T] = [\, v \in t \mid v \,]}{\texttt{cast(t, sequence)} \rightarrow \texttt{S}}$$ (Tuple-to-sequence typecast)

$$\frac{t : (T, ...) \quad B : bag[T] = [\![\, v \in t \mid v \,]\!]}{\texttt{cast(t, bag)} \rightarrow \texttt{B}}$$ (Tuple-to-bag typecast)

$$\frac{t : (T_1, T_2, ...) \quad R : record[T_1, T_2, ...] = R_{type}(\, v \in t \mid v \,) \quad \forall\, i \in 0..|t| \mid type(t[i]) = type(R[i]) \quad |t| = |R|}{\texttt{cast(t, record)} \rightarrow \texttt{R}}$$ (Tuple-to-record typecast)

$$\frac{s : set[T] \quad s_q = \{\, v \in s \mid cast(v, str) \,\} \quad q : \text{“}s_q\text{”}}{\texttt{cast(s, str)} \rightarrow \texttt{q}}$$ (Set-to-string typecast)

$$\frac{s : set[T] \quad S : sequence[T] = [\, v \in s \mid v \,]}{\texttt{cast(s, sequence)} \rightarrow \texttt{S}}$$ (Set-to-sequence typecast)

$$\frac{s : set[T]}{\texttt{cast(s, tuple)} \rightarrow \texttt{cast(cast(s, sequence), tuple)}}$$ (Set-to-tuple typecast)

$$\frac{s : set[T] \quad B : bag[T] = [\![\, v \in s \mid v \,]\!]}{\texttt{cast(s, bag)} \rightarrow \texttt{B}}$$ (Set-to-bag typecast)

$$\frac{s : set[(T_1, T_2)] \quad r : relation[T_1, T_2] = \{\, (v_1, v_2) \in s \mid v_1 \mapsto v_2 \,\}}{\texttt{cast(s, relation)} \rightarrow \texttt{r}}$$ (Set-to-relation typecast)

$$\frac{r : relation[T_1, T_2] \quad r_q = \{\, v_1 \mapsto v_2 \in r \mid cast(v_1, str) \mapsto cast(v_2, str) \,\} \quad q : \text{“}r_q\text{”}}{\texttt{cast(r, str)} \rightarrow \texttt{q}}$$ (Relation-to-string typecast)

$$\frac{r : relation[T_1, T_2] \quad s : set[(T_1, T_2)] = \{\, v_1 \mapsto v_2 \in r \mid (v_1, v_2) \,\}}{\texttt{cast(r, relation)} \rightarrow \texttt{s}}$$ (Relation-to-set typecast)

$$\frac{r : relation[T_1, T_2]}{\texttt{cast(r, sequence)} \rightarrow \texttt{cast(cast(s, set), sequence)}}$$ (Relation-to-sequence typecast)

$$\frac{r : relation[T_1, T_2]}{\texttt{cast(r, sequence)} \rightarrow \texttt{cast(cast(s, sequence), tuple)}}$$ (Relation-to-tuple typecast)

$$\frac{B : bag[T] \quad B_q = [\![\, v \in B \mid cast(v, str) \,]\!] \quad q : \text{“}B_q\text{”}}{\texttt{cast(B, str)} \rightarrow \texttt{q}}$$ (Bag-to-string typecast)

$$\frac{B : bag[T] \quad r : relation[T, int] = \{\, v \mapsto i \in B \mid v \mapsto i \,\}}{\texttt{cast(B, set)} \rightarrow \texttt{r}}$$ (Bag-to-relation typecast)

$$\frac{B : bag[T] \quad s : set[T] = \{\, v \in B \mid v \,\}}{\texttt{cast(B, set)} \rightarrow \texttt{s}}$$ (Bag-to-set typecast)

$$\frac{B : bag[T] \quad S : sequence[T] = flatten([\, v \mapsto i \in B \mid [\, j \in 0\,..\,i \mid v \,]\,])}{\texttt{cast(B, sequence)} \rightarrow \texttt{S}}$$ (Bag-to-sequence typecast)

$$\frac{B : bag[T]}{\texttt{cast(B, tuple)} \rightarrow \texttt{cast(cast(B, sequence), tuple)}}$$ (Bag-to-tuple typecast)

$$\frac{S : sequence[T] \quad S_q = [\, v \in S \mid cast(v, str) \,] \quad q : \text{“}S_q\text{”}}{\texttt{cast(S, str)} \rightarrow \texttt{q}}$$ (Sequence-to-string typecast)

$$\frac{S : sequence[T] \quad s : set[T] = \{\, v \in S \mid v \,\}}{\texttt{cast(S, set)} \rightarrow \texttt{s}} \quad \text{(Sequence-to-set typecast)}$$

$$\frac{S : sequence[T] \quad r : relation[int, T] = \{\, i \mapsto v \in S \mid i \mapsto v \,\}}{\texttt{cast(S, relation)} \rightarrow \texttt{r}} \quad \text{(Sequence-to-relation typecast)}$$

$$\frac{S : sequence[T] \quad t : (T, ...) = (\, v \in S \mid v \,)}{\texttt{cast(S, tuple)} \rightarrow \texttt{t}} \quad \text{(Sequence-to-tuple typecast)}$$

$$\frac{S : sequence[T]}{\texttt{cast(S, bag)} \rightarrow \texttt{cast(cast(S, tuple), bag)}} \quad \text{(Sequence-to-bag typecast)}$$

$$\frac{R : record[T_1, T_2, ...] \quad R_q = \{\, field \mapsto v \in R \mid field \mapsto cast(v, str) \,\} \quad q : \text{“}R_q\text{”}}{\texttt{cast(R, str)} \rightarrow \texttt{q}} \quad \text{(Record-to-string typecast)}$$

$$\frac{R : record[T, ...] \quad r : relation[str, T] = \{\, field \mapsto v \in R \mid field \mapsto v \,\}}{\texttt{cast(R, relation)} \rightarrow \texttt{r}} \quad \text{(Record-to-relation typecast)}$$

$$\frac{R : record[T, ...]}{\texttt{cast(R, set)} \rightarrow \texttt{cast(cast(R, relation), set)}} \quad \text{(Record-to-set typecast)}$$

$$\frac{R : record[T, ...]}{\texttt{cast(R, sequence)} \rightarrow \texttt{cast(cast(R, tuple), sequence)}} \quad \text{(Record-to-sequence typecast)}$$

$$\frac{R : record[T_1, T_2, ...] \quad t : (T_1, T_2, ...) = (\, field \mapsto v \in R \mid v \,)}{\texttt{cast(R, tuple)} \rightarrow \texttt{t}} \quad \text{(Record-to-tuple typecast)}$$

Equals For primitives, equality is calculated in the usual mathematical manner. We explicitly specify equality for collection types, although these also follow convention.

$$\frac{T \in \{\, none, bool, int, float, str, enum \,\} \quad v_1, v_2 : T \quad b : bool = (v_1 = v_2)}{\texttt{v_1 = v_2} \rightarrow \texttt{b}} \quad \text{(Primitives)}$$

$$\frac{t_1 : (T_1, T_2, ...) \quad t_2 : (T_1, T_2, ...) \quad |t_1| = |t_2| \quad b : bool = \forall\, i \in 0..|t_1| \mid t_1[i] = t_2[i]}{\texttt{t_1 = t_2} \rightarrow \texttt{b}} \quad \text{(Tuple)}$$

$$\frac{s_1 : set[T] \quad s_2 : set[T] \quad |s_1| = |s_2| \quad b : bool = \forall\, v \in s_1 \mid v \in s_2}{\texttt{s_1 = s_2} \rightarrow \texttt{b}} \quad \text{(Set)}$$

$$\frac{r_1 : relation[T_1, T_2] \quad r_2 : relation[T_1, T_2] \quad |r_1| = |r_2| \quad b : bool = \forall\, v \in dom(r_1) \mid r_1[v] = r_2[v]}{\texttt{r_1 = r_2} \rightarrow \texttt{b}} \quad \text{(Relation)}$$

$$\frac{b_1 : bag[T] \quad b_2 : bag[T]}{\texttt{b_1 = b_2} \rightarrow \texttt{cast(b_1, relation) = cast(b_2, relation)}} \quad \text{(Bag)}$$

$$\frac{S_1 : sequence[T] \quad S_2 : sequence[T]}{\texttt{S_1 = S_2} \rightarrow \texttt{cast(S_1, relation) = cast(S_2, relation)}} \quad \text{(Sequence)}$$

$$\frac{R_1 : record[T_1, T_2, ...] \quad R_2 : record[T_1, T_2, ...] \quad b : bool = \forall\, f \in dom(R_1) \mid R_1.f = R_2.f}{\texttt{R_1 = R_2} \rightarrow \texttt{b}} \quad \text{(Record)}$$

Not equals

$$\frac{v_1, v_2 : T \quad b : bool = \neg(v_1 = v_2)}{\texttt{v_1 != v_2} \rightarrow \texttt{b}} \quad \text{(Not equals)}$$

Bool

Not

$$\frac{b_1, b_2 : \mathit{bool} \quad b_2 = \neg b_1}{\texttt{not b_1} \to \texttt{b_2}} \tag{Not}$$

Equivalent Similar to equals, but lower precedence and only for booleans.

$$\frac{b_1, b_2, b_3 : \mathit{bool} \quad b_3 : \mathit{bool} = (b_1 \equiv b_2)}{\texttt{b_1 == b_2} \to \texttt{b_3}} \tag{Equivalent}$$

Not equivalent

$$\frac{b_1, b_2, b_3 : \mathit{bool} \quad b_3 : \mathit{bool} = (b_1 \not\equiv b_2)}{\texttt{b_1 !== b_2} \to \texttt{b_3}} \tag{Not equivalent}$$

Implies

$$\frac{b_1, b_2, b_3 : \mathit{bool} \quad b_3 : \mathit{bool} = (b_1 \implies b_2)}{\texttt{b_1 => b_2} \to \texttt{b_3}} \tag{Implies}$$

And To enable the optimizer to shuffle around $\wedge$-conditions within programs following discrete maths, we cannot use the programming-specific short-circuiting features. Conventional short-circuiting evaluates conditions from left to right, stopping at the first annihilating clause (false for $\wedge$, true for $\vee$). However, this can hide potentially $\bot$-valued clauses to the right of the annihilator. So, when the optimizer leans on commutativity to reorder clauses, behaviour would not be guaranteed. Optimistic short-circuiting keeps the same idea as short-circuiting, but extends it to recover from $\bot$-valued clauses when an annihilator is present in any clause. We only pay an efficiency penalty in the case where we would otherwise throw an error.

$$\frac{b_1, b_2, b_3 : \mathit{bool} \quad b_3 = (b_1 \wedge b_2) = (b_2 \wedge b_1)}{\texttt{b_1 and b_2} \to \texttt{b_3}} \tag{And}$$

$$\frac{b : \mathit{bool} \quad v \in \{\bot, b\}}{\texttt{false and v} \to \texttt{false}} \tag{Optimistic short-circuiting (left)}$$

$$\frac{b : \mathit{bool} \quad v \in \{\bot, b\}}{\texttt{v and false} \to \texttt{false}} \tag{Optimistic short-circuiting (right)}$$

Or Similar optimistic short-circuiting as the And conditions above.

$$\frac{b_1, b_2, b_3 : \mathit{bool} \quad b_3 = (b_1 \vee b_2) = (b_2 \vee b_1)}{\texttt{b_1 or b_2} \to \texttt{b_3}} \tag{Or}$$

$$\frac{b : \mathit{bool} \quad v \in \{\bot, b\}}{\texttt{true or v} \to \texttt{true}} \tag{Optimistic short-circuiting (left)}$$

$$\frac{b : \mathit{bool} \quad v \in \{\bot, b\}}{\texttt{v or true} \to \texttt{true}} \tag{Optimistic short-circuiting (right)}$$

Arithmetic Integers follow unbounded precision arithmetic, while floats follow IEEE-754. Addition is commutative for pure-int operations (and thus can be used in the optimizer more freely), while floats remain bound by the written order because of precision limitations. Multiplication and exponentiation are semantically defined in terms of addition, and thus follow similar semantics: close to mathematics for ints, close to programming for floats.

Less than

$$\frac{T_1 \in \{\, int, float \,\} \quad T_2 \in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad b : bool = (v_1 < v_2)}{\texttt{v_1 < v_2} \to \texttt{b}} \quad \text{(Less than)}$$

Greater than

$$\frac{T_1 \in \{\, int, float \,\} \quad T_2 \in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad b : bool = (v_1 > v_2)}{\texttt{v_1 > v_2} \to \texttt{b}} \quad \text{(Greater than)}$$

Less than or equals

$$\frac{T_1 \in \{\, int, float \,\} \quad T_2 \in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad b : bool = (v_1 \leq v_2)}{\texttt{v_1 <= v_2} \to \texttt{b}} \quad \text{(Less than or equals)}$$

Greater than or equals

$$\frac{T_1 \in \{\, int, float \,\} \quad T_2 \in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad b : bool = (v_1 \geq v_2)}{\texttt{v_1 >= v_2} \to \texttt{b}} \quad \text{(Greater than or equals)}$$

Negate

$$\frac{i_1 : int \quad i_2 : int = (-i_1)}{\texttt{-i_1} \to \texttt{i_2}} \quad \text{(Int)}$$

$$\frac{f_1 : float \quad f_2 : float = (-f_1)}{\texttt{-f_1} \to \texttt{f_2}} \quad \text{(Float)}$$

Int divide

$$\frac{i_1 : int \quad i_2 : int \quad i_3 : int = \lfloor \frac{i_1}{i_2} \rfloor}{\texttt{i_1 div i_2} \to \texttt{i_3}} \quad \text{(Int divide)}$$

Modulo

$$\frac{i_1 : int \quad i_2 : int \quad i_3 : int = i_1 - i_2 \times \lfloor \frac{i_1}{i_2} \rfloor}{\texttt{i_1 mod i_2} \to \texttt{i_3}} \quad \text{(Modulo)}$$

Add

$$\frac{i_1, i_2, i_3 : int \quad i_3 = (i_1 + i_2) = (i_2 + i_1)}{\texttt{i_1 + i_2} \to \texttt{i_3}} \quad \text{(Int and Int)}$$

$$\frac{i : int \quad f_1, f_2 : float \quad f_2 = i + f_1}{\texttt{i + f_1} \to \texttt{f_2}} \quad \text{(Int and Float)}$$

$$\frac{i : int \quad f_1, f_2 : float \quad f_2 = f_1 + i}{\texttt{f_1 + i} \rightarrow \texttt{f_2}} \quad \text{(Float and Int)}$$

$$\frac{f_1, f_2, f_3 : float \quad f_3 = f_1 + f_2}{\texttt{f_1 + f_2} \rightarrow \texttt{f_3}} \quad \text{(Float and Float)}$$

Subtract

$$\frac{T_1, T_2, T_3 :\in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad v_3 : T_3 \quad v_3 = v_1 + -(v_2)}{\texttt{v_1 - v_2} \rightarrow \texttt{v_3}} \quad \text{(Subtract)}$$

Divide

$$\frac{T_1, T_2 :\in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad f : float \quad f = \frac{v_1}{v_2}}{\texttt{v_1 / v_2} \rightarrow \texttt{f}} \quad \text{(Subtract)}$$

Multiply

$$\frac{i_1, i_2, i_3 : int \quad i_3 = \sum i \in 0 \,..\, i_1 \mid i_2}{\texttt{i_1 * i_2} \rightarrow \texttt{i_3}} \quad \text{(Int and Int)}$$

$$\frac{i : int \quad f_1, f_2 : float \quad f_2 = i \times f_1}{\texttt{i * f_1} \rightarrow \texttt{f_2}} \quad \text{(Int and Float)}$$

$$\frac{i : int \quad f_1, f_2 : float \quad f_2 = f_1 \times i}{\texttt{f_1 * i} \rightarrow \texttt{f_2}} \quad \text{(Float and Int)}$$

$$\frac{f_1, f_2, f_3 : float \quad f_3 = f_1 \times f_2}{\texttt{f_1 * f_2} \rightarrow \texttt{f_3}} \quad \text{(Float and Float)}$$

Power

$$\frac{T_1, T_2 :\in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad i : int \quad v_2 = \prod i \in 0 \,..\, i \mid v_1}{\texttt{v_1 \^{} i} \rightarrow \texttt{v_2}} \quad \text{(Int exponent)}$$

$$\frac{T_1, T_2 :\in \{\, int, float \,\} \quad v_1 : T_1 \quad v_2 : T_2 \quad f : float \quad v_2 = v_1^f}{\texttt{v_1 \^{} f} \rightarrow \texttt{v_2}} \quad \text{(Float exponent)}$$

Upto

$$\frac{i_1, i_2 : int \quad s : set[int] \quad s = i_1 \,..\, i_2}{\texttt{i_1 .. i_2} \rightarrow \texttt{s}} \quad \text{(Upto)}$$

String

Tuple

Enumeration

$$\frac{v_1 : T_1 \quad v_2 : T_2 \quad t : (T_1, T_2, ...) = (\, v_1, v_2, ... \,)}{\texttt{(v_1, v_2, ...)} \rightarrow \texttt{t}} \quad \text{(Enumeration)}$$

Maplet

$$\frac{v_1 : T_1 \quad v_2 : T_2 \quad t : (T_1, T_2) = (\, v_1, v_2 \,)}{\texttt{v_1 |-> v_2} \rightarrow \texttt{t}} \quad \text{(Maplet)}$$

Record

Definition Records are represented internally as a relation from strings to any field-independent type.

$$\frac{i : int \in dom(\rho)}{\langle \texttt{record R: skip}, \sigma, \rho \rangle \rightarrow \langle \texttt{skip}, \sigma \oplus \{\, R \mapsto i \,\}, \rho \oplus \{\, i \mapsto type[record\{\ \}] \,\} \rangle} \quad \text{(Empty)}$$

$$\frac{i : int \in dom(\rho)}{\langle \texttt{record R: (f_1: T_1, f_2: T_2, ...)}, \sigma, \rho \rangle \rightarrow \langle , \sigma \oplus \{\, R \mapsto i \,\}, \rho \oplus \{\, i \mapsto type[record\{\, f_1 \mapsto T_1, f_2 \mapsto T_2 \,\}] \,\} \rangle} \quad \text{(Fields)}$$

Access

$$\frac{R_{type} : record\{\, f_1 \mapsto T_1, f_2 \mapsto T_2 \,\} \in \sigma \circ \rho \quad R = R_{type}(v_1, v_2, ...)}{\texttt{R.f_n} \rightarrow \texttt{v_n}} \quad \text{(Access)}$$

Set

Enumeration

$$\frac{v_1, v_2 : T \quad s : set[T] = \{\, v_1, v_2, ... \,\}}{\texttt{\{ v_1, v_2, ... \}} \rightarrow \texttt{s}} \quad \text{(Enumeration)}$$

Clear

$$\frac{s_1, s_2 : set[T] \quad s_2 = \{\ \}}{\texttt{clear(s_1)} \rightarrow \texttt{s_2}} \quad \text{(Clear)}$$

Add

$$\frac{s_1, s_2 : set[T] \quad v : T \quad s_2 = s_1 \cup \{\, v \,\}}{\texttt{add(s_1)} \rightarrow \texttt{s_2}} \quad \text{(Add)}$$

Remove

$$\frac{s_1, s_2 : set[T] \quad v : T \quad s_2 = s_1 \setminus \{\, v \,\}}{\texttt{remove(s_1, v)} \rightarrow \texttt{s_2}} \quad \text{(Remove)}$$

In

$$\frac{s : set[T] \quad v : T \quad b : bool = v \in s}{\texttt{v in s} \rightarrow \texttt{b}} \quad \text{(In)}$$

Not in

$$\frac{s : set[T] \quad v : T \quad b : bool = v \notin s}{\texttt{v not in s} \rightarrow \texttt{b}} \quad \text{(Not In)}$$

Cardinality

$$\frac{s : set[T] \quad i : int = |s|}{\texttt{card(s)} \rightarrow \texttt{i}} \quad \text{(Cardinality)}$$

Choice

$$\frac{s : set[T] \quad v : T \quad v \in s}{\texttt{choice(s)} \rightarrow \texttt{v}} \quad \text{(Normal execution)}$$

$$\frac{s : set[T] \quad s = \{\ \} \quad v = \bot}{\texttt{choice(s)} \rightarrow \texttt{v}} \quad \text{(Undefined)}$$

Forall

$$\frac{s : set[bool] \quad b = \forall e \in s}{\texttt{forall(s)} \rightarrow \texttt{b}} \quad \text{(Forall)}$$

Exists

$$\frac{s : set[bool] \quad b = \exists e \in s}{\texttt{exists(s)} \rightarrow \texttt{b}} \quad \text{(Exists)}$$

Min

$$\frac{T :\in \{ int, float, str, tuple[T] \} \quad s : set[T] \quad v : T \quad v \in s \quad \forall e \in s \mid v \leq e}{\texttt{min(s)} \rightarrow \texttt{v}} \quad \text{(Min)}$$

Max

$$\frac{T :\in \{ int, float, str, tuple[T] \} \quad s : set[T] \quad v : T \quad v \in s \quad \forall e \in s \mid v \geq e}{\texttt{max(s)} \rightarrow \texttt{v}} \quad \text{(Max)}$$

Sum

$$\frac{T :\in \{ int, float \} \quad s : set[T] \quad v : T = \sum e \in s \mid e}{\texttt{sum(s)} \rightarrow \texttt{v}} \quad \text{(Sum)}$$

Product

$$\frac{T :\in \{ int, float \} \quad s : set[T] \quad v : T = \prod e \in s \mid e}{\texttt{product(s)} \rightarrow \texttt{v}} \quad \text{(Product)}$$

General Union

$$\frac{s : set[set[T]] \quad v : set[T] = \bigcup e \in s \mid e}{\texttt{union(s)} \rightarrow \texttt{v}} \quad \text{(Union)}$$

General Intersection

$$\frac{s : set[set[T]] \quad v : set[T] = \bigcap e \in s \mid e}{\texttt{intersection(s)} \rightarrow \texttt{v}} \quad \text{(Intersection)}$$

Map

$$\frac{s : set[T_1] \quad p : procedure[T_1, T_2] \quad v = \{\, xins \mid p(x) \,\}}{\texttt{map(p, s)} \rightarrow \texttt{v}} \quad \text{(Map)}$$

Map min

$$\frac{s : set[T] \quad p : procedure[T, int] \quad v_1 = \{\, xins \mid p(x) \mapsto x \,\} \quad v_2 = v_1(min(dom(v_1)))}{\texttt{map_min(p, s)} \rightarrow \texttt{v_2}} \quad \text{(Map min)}$$

Map max

$$\frac{s : set[T] \quad p : procedure[T, int] \quad v_1 = \{\, xins \mid p(x) \mapsto x \,\} \quad v_2 = v_1(max(dom(v_1)))}{\texttt{map_max(p, s)} \rightarrow \texttt{v_2}} \quad \text{(Map max)}$$

Union

$$\frac{s_1, s_2, s_3 : set[T] \quad s_3 = s_1 \cup s_2}{\texttt{s_1 \textbackslash/ s_2} \rightarrow \texttt{v}} \quad \text{(Union)}$$

Intersection

$$\frac{s_1, s_2, s_3 : set[T] \quad s_3 = s_1 \cap s_2}{\texttt{s_1 /\textbackslash\ s_2} \rightarrow \texttt{v}} \quad \text{(Intersection)}$$

Difference

$$\frac{s_1, s_2, s_3 : set[T] \quad s_3 = s_1 \setminus s_2}{\texttt{s_1 \textbackslash\ s_2} \rightarrow \texttt{v}} \quad \text{(Difference)}$$

Cartesian product

$$\frac{s_1 : set[T_1] \quad s_2 : set[T_2] \quad r : relation[T_1, T_2] = \{\, v_1 \in s_1 \wedge v_2 \in s_2 \mid v_1 \mapsto v_2 \,\}}{\texttt{s_1 >< s_2} \rightarrow \texttt{r}} \quad \text{(Cartesian product)}$$

Subset

$$\frac{s_1, s_2 : set[T] \quad b : bool \quad b = \forall\, v_1 \in s_1 \mid v_1 \in s_2 \wedge |s_1| < |s_2|}{\texttt{s_1 <:\ \ s_2} \rightarrow \texttt{b}} \quad \text{(Subset)}$$

Subset equals

$$\frac{s_1, s_2 : set[T] \quad b : bool \quad b = \forall\, v_1 \in s_1 \mid v_1 \in s_2}{\texttt{s_1 <:\ \ s_2} \rightarrow \texttt{b}} \quad \text{(Subset equals)}$$

Superset

$$\frac{s_1, s_2 : set[T]}{\texttt{s_1 :>> s_2} \rightarrow \texttt{s_2 <<: s_1}} \quad \text{(Superset)}$$

Superset equals

$$\frac{s_1, s_2 : set[T]}{\texttt{s_1 :> s_2} \rightarrow \texttt{s_2 <: s_1}} \quad \text{(Superset equals)}$$

Not subset

$$\frac{s_1, s_2 : set[T]}{\texttt{s_1 !<<: s_2} \rightarrow \texttt{!(s_1 <<: s_2)}} \quad \text{(Not subset)}$$

Not subset equals

$$\frac{s_1, s_2 : set[T]}{\texttt{s_1 !<: s_2} \rightarrow \texttt{!(s_1 <: s_2)}} \quad \text{(Not subset equals)}$$

Not superset

$$\frac{s_1, s_2 : set[T]}{\texttt{s_1 !:>> s_2} \rightarrow \texttt{!(s_1 :>> s_2)}} \quad \text{(Not superset)}$$

Not superset equals

$$\frac{s_1, s_2 : set[T]}{\texttt{s_1 !:> s_2} \rightarrow \texttt{!(s_1 :> s_2)}} \quad \text{(Not superset equals)}$$

Relation

Inverse

$$\frac{r_1 : relation[T_1, T_2] \quad r_2 : relation[T_2, T_1] = \{\, v_1 \mapsto v_2 \in r_1 \mid v_2 \mapsto v_1 \,\}}{\texttt{r_1 ~ -1} \rightarrow \texttt{r_2}} \quad \text{(Inverse)}$$

Function call

$$\frac{r : relation[T_1, T_2] \quad v : T_1}{\texttt{r(v)} \rightarrow \texttt{choice(r[\{ v \}])}} \quad \text{(Function call)}$$

Image

$$\frac{r : relation[T_1, T_2] \quad v : T_1 \quad s : set[T_2] = \{\, v_1 \mapsto v_2 \in r \cdot v_1 = v \mid v_2 \,\}}{\texttt{r[v]} \rightarrow \texttt{s}} \quad \text{(Image)}$$

Overriding

$$\frac{r_1, r_2, r_3 : relation[T_1, T_2] \quad r_3 = r_2 \cup (dom(r_2) \lhd r_1)}{\texttt{r_1 <+ r_2} \rightarrow \texttt{r_3}} \text{(Overriding)}$$

Domain

$$\frac{r : relation[T_1, T_2] \quad s : set[T_1] = \{\, v_1 \mapsto v_2 \in r \mid v_1 \,\}}{\texttt{dom(r)} \rightarrow \texttt{s}} \text{(Domain)}$$

Range

$$\frac{r : relation[T_1, T_2] \quad s : set[T_1] = \{\, v_1 \mapsto v_2 \in r \mid v_2 \,\}}{\texttt{ran(r)} \rightarrow \texttt{s}} \text{(Range)}$$

Domain restriction

$$\frac{r_1, r_2 : relation[T_1, T_2] \quad s : set[T_1] \quad r_2 = \{\, v_1 \mapsto v_2 \in r \cdot v_1 \in s \mid v_1 \mapsto v_2 \,\}}{\texttt{s <| r_1} \rightarrow \texttt{r_2}} \text{(Domain restriction)}$$

Domain subtraction

$$\frac{r_1, r_2 : relation[T_1, T_2] \quad s : set[T_1] \quad r_2 = \{\, v_1 \mapsto v_2 \in r \cdot v_1 \notin s \mid v_1 \mapsto v_2 \,\}}{\texttt{s <<| r_1} \rightarrow \texttt{r_2}} \text{(Domain subtraction)}$$

Range restriction

$$\frac{r_1, r_2 : relation[T_1, T_2] \quad s : set[T_1] \quad r_2 = \{\, v_1 \mapsto v_2 \in r \cdot v_2 \in s \mid v_1 \mapsto v_2 \,\}}{\texttt{r_1 |> s} \rightarrow \texttt{r_2}} \text{(Range restriction)}$$

Range subtraction

$$\frac{r_1, r_2 : relation[T_1, T_2] \quad s : set[T_1] \quad r_2 = \{\, v_1 \mapsto v_2 \in r \cdot v_2 \notin s \mid v_1 \mapsto v_2 \,\}}{\texttt{r_1 |>> s} \rightarrow \texttt{r_2}} \text{(Range subtraction)}$$

Composition

$$\frac{r_1 : relation[T_1, T_2] \quad r_2 : relation[T_2, T_3] \quad r_3 : relation[T_1, T_3] = \{\, v_1 \mapsto v_2 \in r_1 \wedge v_3 \mapsto v_4 \in r_2 \cdot v_2 = v_3 \mid v_1 \mapsto v_4 \,\}}{\texttt{r_1 <> r_2} \rightarrow \texttt{r_3}} \text{(Composition)}$$

Bag Image

$$\frac{r : relation[T_1, T_2] \quad B_1 : bag[T_1] \quad B_2 : bag[T_2] \quad B_2 = r^{-1} \circ B_1}{\texttt{r[B_1]} \rightarrow \texttt{B_2}} \text{(Bag image)}$$

Bag

Bag union

$$\frac{B_1, B_2, B_3 : bag[T] \qquad B_3 = [\![\, v \in (dom(B_1) \cup dom(B_2)) \cdot i = max(B_1[v] \cup B_2[v]) \mid v \mapsto i \,]\!]}{\texttt{B_1 \textbackslash/ B_2} \rightarrow \texttt{B_3}} \quad \text{(Bag union)}$$

Bag intersection

$$\frac{B_1, B_2, B_3 : bag[T] \qquad B_3 = [\![\, v \in dom(B_1) \cdot v \in dom(B_2) \wedge i = min(B_1[v] \cup B_2[v]) \wedge i > 0 \mid v \mapsto i \,]\!]}{\texttt{B_1 /\textbackslash\ B_2} \rightarrow \texttt{B_3}} \quad \text{(Bag intersection)}$$

Bag add

$$\frac{B_1, B_2, B_3 : bag[T] \qquad B_3 = [\![\, v \in (dom(B_1) \cup dom(B_2)) \cdot i = sum(B_1[v] \cup B_2[v]) \mid v \mapsto i \,]\!]}{\texttt{B_1 + B_2} \rightarrow \texttt{B_3}} \quad \text{(Bag add)}$$

Bag difference

$$\frac{B_1, B_2, B_3 : bag[T] \qquad B_3 = [\![\, v \in dom(B_1) \cdot i = B_1(v) - (B_2 \cup [\![\, v \mapsto 0 \,]\!])(v) \wedge i > 0 \mid v \mapsto i \,]\!]}{\texttt{B_1 - B_2} \rightarrow \texttt{B_3}} \quad \text{(Bag difference)}$$

Size

$$\frac{B : bag[T] \qquad i : int = \sum v \mapsto i_1 \mid i_1}{\texttt{size(B)} \rightarrow \texttt{i}} \quad \text{(Size)}$$

Sequence

Concat

$$\frac{S_1, S_2, S_3 : sequence[T] \qquad S_3 = S_1 \cup \{\, i \mapsto v \in S_2 \mid i + max(dom(S_1)) \mapsto v \,\}}{\texttt{S_1 ++ S_2} \rightarrow \texttt{S_3}} \quad \text{(Concat)}$$

Procedure

Definition

$$\frac{i : int \notin dom(\rho) \qquad T_1, ..., T_n \in dom(\rho) \qquad T \in dom(\rho) \qquad args = \{\, a_1 \mapsto T_1, ..., a_n \mapsto T_n \,\}}{\langle \texttt{procedure p(a_1: t_1, ..., a_n: t_n) -> T:}\ block(\texttt{C}) \rangle \rightarrow \langle \texttt{skip}, \sigma \oplus \{\, p \mapsto i \,\}, \rho \oplus \{\, i \mapsto procedure(args, T, block(C), \sigma, \rho)}$$
(Procedure Definition)

Call

$$\frac{p \mapsto procedure(args, T, block(C), \sigma_p, \rho_p) \in \sigma \circ \rho \qquad a_1 \mapsto t_1, ..., a_n \mapsto t_n}{\langle \texttt{p(v_1, ..., v_n)}, \sigma, \rho \rangle \rightarrow \langle block(\texttt{a_1 := v_1; ...; a_n := v_n; C}); restore(\sigma, \rho), \sigma_p, \rho_p \rangle} \quad \text{(Procedure call)}$$

Quantifications

Min

$$\frac{}{\texttt{min x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{min}(\{\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,\})} \quad \text{(Min)}$$

Max

$$\frac{}{\texttt{max x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{max}(\{\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,\})} \quad \text{(Max)}$$

Forall

$$\frac{}{\texttt{forall x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{forall}(\{\, \texttt{x in S} \mid \texttt{P and E} \,\})} \quad \text{(Forall)}$$

Exists

$$\frac{}{\texttt{exists x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{exists}(\{\, \texttt{x in S} \mid \texttt{P and E} \,\})} \quad \text{(Exists)}$$

Union all

$$\frac{}{\texttt{union x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{union}(\{\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,\})} \quad \text{(Union)}$$

Intersection all

$$\frac{}{\texttt{intersection x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{intersection}(\{\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,\})} \quad \text{(Intersection)}$$

Sum

$$\frac{}{\texttt{sum x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{sum}([\![\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,]\!])} \quad \text{(Sum)}$$

Product

$$\frac{}{\texttt{product x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{product}([\![\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,]\!])} \quad \text{(Product)}$$

Comprehension

$$\frac{s = \{\, x \in S \cdot P \mid E \,\}}{\{\, \texttt{x in S} \cdot \texttt{P} \mid \texttt{E} \,\} \rightarrow s} \quad \text{(Comprehension)}$$

$$\frac{s = \{\, (x_1, ..., x_n) \in S_1 \times ... \times S_n \cdot P \mid E \,\}}{\{\, \texttt{x_1 in S_1, ..., x_n in S_n} \cdot \texttt{P} \mid \texttt{E} \,\} \rightarrow s} \quad \text{(Nested)}$$

$$\frac{s = \{\, x_1 \in S_1 \cdot P_1 \mid E_1 \,\} \cup ... \cup \{\, x_n \in S_n \cdot P_n \mid E_n \,\}}{\{\, \texttt{x_1 in S_1} \cdot \texttt{P_1} \mid \texttt{E_1 `} \ ... \ \texttt{` x_n in S_n} \cdot \texttt{P_n} \mid \texttt{E_n} \,\} \rightarrow s} \quad \text{(Joined)}$$

Lambda

Iter

$$\frac{t = (\, i \in 0..n \mid a_i \,) \qquad S \neq \{\ \}}{\texttt{iter x in S} \cdot \texttt{P : a_1 := x_1; ...; a_n := x_n} \rightarrow \texttt{t} \mid \mathit{block}(\texttt{C}) \rightarrow \mathit{block}(\texttt{x := choice(S); a_1 := x_1; ...; a_n := x_n; if}} \quad \text{(Fold)}$$

Fold

$$\frac{}{\texttt{fold v := i : x in S} \cdot \texttt{P} \mid \texttt{E} \rightarrow \texttt{v := i; for x in S: } \mathit{block}(\texttt{if P: v := E})} \quad \text{(Fold)}$$

Simple Statements

Assignment

$$\frac{i \notin dom(\rho)}{\langle \texttt{v_1 := v_2}, \sigma, \rho \rangle \rightarrow \langle \texttt{skip}, \sigma \oplus \{\, v_1 \mapsto i \,\}, \rho \oplus \{\, i \mapsto v_2 \,\} \rangle} \quad \text{(Assignment)}$$

$$\frac{v_1 \mapsto v_2 \in \sigma \circ \rho}{\langle \texttt{v_1}, \sigma, \rho \rangle \rightarrow \langle \texttt{v_2}, \sigma, \rho \rangle} \quad \text{(Dereferencing)}$$

Trait Application

$$\frac{i \notin dom(\rho)}{\langle \texttt{v_1 := v_2 trait v_3}, \sigma, \rho \rangle \rightarrow \langle \texttt{skip}, \sigma \oplus \{\, v_1 \mapsto i \,\}, \rho \oplus \{\, i \mapsto \mathit{traits}(v_2, v_3) \,\} \rangle} \quad \text{(Trait Assignment)}$$

Break

$$\frac{}{\texttt{block(C_1;break;C_2)} \rightarrow \texttt{block(C_1)}} \quad \text{(Break)}$$

Continue

$$\frac{}{\texttt{loop(C_1;continue;C_2)} \rightarrow \texttt{loop(C_1)}} \quad \text{(Break)}$$

Import Statements

$$\frac{q_{file} = [0-9A-Za-z_]+ \qquad q = q_{file_0} \backslash ... \backslash q_{file_n} \qquad i \notin dom(\rho) \qquad \sigma_q = parse(q)(\sigma) \qquad \rho_q = parse(q)(\rho)}{\langle \texttt{import q}, \sigma, \rho \rangle \rightarrow \langle \mathit{skip}, \sigma \oplus \{\, q_{file_n} \mapsto \sigma_q \,\}, \rho \oplus \{\, q_{file_n} \mapsto \rho_q \,\} \rangle} \quad \text{(Module Import)}$$

$$\frac{q_{file} = [0-9A-Za-z_]+ \qquad q = q_{file_0} \backslash ... \backslash q_{file_n} \qquad i \notin dom(\rho) \qquad \sigma_q = parse(q)(\sigma) \qquad \rho_q = parse(q)(\rho)}{\langle \texttt{from q import *}, \sigma, \rho \rangle \rightarrow \langle \mathit{skip}, \sigma \oplus \sigma_q, \rho \oplus \rho_q \rangle} \quad \text{(Unnamed Module Import)}$$

$$\frac{q_{file} = [0-9A-Za-z_]+ \qquad q = q_{file_0} \backslash ... \backslash q_{file_n} \qquad i \notin dom(\rho) \qquad \sigma_q = parse(q)(\sigma) \qquad \rho_q = parse(q)(\rho)}{\langle \texttt{from q import a_1, ..., a_n}, \sigma, \rho \rangle \rightarrow \langle \mathit{skip}, \sigma \oplus \{\, a_1 \mapsto \sigma_q(a_1), ..., a_n \mapsto \sigma_q(a_n) \,\}, \rho \oplus \{\, a_1 \mapsto \rho_q(a_1), ..., a_n \mapsto \rho_q(a_n) \,\} \rangle} \quad \text{(Named Import)}$$

Compound Statements

If

$$\frac{b = true}{\texttt{if b:}\quad block(\texttt{C}) \rightarrow block(\texttt{C})} \quad \text{(If (true))}$$

$$\frac{b = false}{\texttt{if b:}\quad block(\texttt{C}) \rightarrow \texttt{skip}} \quad \text{(If (false))}$$

If-Else

$$\frac{b = true}{\texttt{if b:}\quad block(\texttt{C_1})\ \texttt{else:}\quad block(\texttt{C_2}) \rightarrow block(\texttt{C_1})} \quad \text{(If-Else (true))}$$

$$\frac{b = false}{\texttt{if b:}\quad block(\texttt{C_1})\ \texttt{else:}\quad block(\texttt{C_2}) \rightarrow block(\texttt{C_2})} \quad \text{(If-Else (false))}$$

For

$$\frac{S = \{\ \}}{\texttt{for x in S:}\ block(\texttt{C}) \rightarrow skip} \quad \text{(For (empty))}$$

$$\frac{S \neq \{\ \}}{\texttt{for x in S:}\ block(\texttt{C}) \rightarrow block(\texttt{v := choice(S); C; for x in S:}\ block(\texttt{C}))} \quad \text{(For)}$$

While

$$\frac{b = false}{\langle \texttt{while b:}\quad block(\texttt{C}), \sigma, \rho\rangle \rightarrow \langle skip, \sigma, \rho\rangle} \quad \text{(While (false))}$$

$$\frac{b = true}{\langle \texttt{while b:}\quad block(\texttt{C}), \sigma, \rho\rangle \rightarrow \langle block(\texttt{C; while b:}\quad block(C)), \sigma, \rho\rangle} \quad \text{(While (true))}$$

Block

$$\frac{}{\langle block(\texttt{C}), \sigma, \rho\rangle \rightarrow \langle \texttt{C;}restore(\sigma), \sigma, \rho\rangle} \quad \text{(Block)}$$

$$\frac{}{\langle restore(\sigma_0), \sigma, \rho\rangle \rightarrow \langle \texttt{skip}, \sigma_0, \rho\rangle} \quad \text{(Restore)}$$

$$\frac{}{\langle \texttt{return v;}restore(\sigma_0), \sigma, \rho\rangle \rightarrow \langle \texttt{v}, \sigma_0, \rho\rangle} \quad \text{(Restore and return)}$$

3 (Abstract) Data Types

3.1 Traits

3.2 Boolean

3.3 Integer

3.4 Float

3.5 String

3.6 Set

3.7 Relation

3.8 Sequence

3.9 Bag

3.10 Tuple

3.11 Record

3.12 Procedure

3.13 Generic

3.13.1 Operations

4 User-Facing Specification

4.1 (Abstract) Data Types

For the most part, Simile's semantics resemble those of set theory and specification languages (specifically, Event-B). Aside from a few primitive types that abstract over bare set theory and programming language specific structures, like integer arithmetic and procedure definitions, all objects inherit properties from sets. For example, a sequence can be modelled as a set of pairs from natural numbers to the sequence element type.

In the forthcoming sections, we explicitly state the design of these set-theory-inspired types, useful properties, and underlying implementations.

4.1.1 Primitives

With consideration to execution efficiency and the extensive amount of existing optimization in modern languages, Simile makes use of the following basic types:

Booleans As the cornerstone of logical reasoning, booleans deserve an explicit type with reserved identifiers: $\mathbb{B} = \{ true, false \}$. Further, Simile provides a healthy supply of common operations beyond $\wedge$ and $\vee$: $\implies, \equiv, \impliedby, \forall, \exists$, etc.

Integers While integers are a very mature type in the world of computing, trade-offs between arbitrary-precision and fixed-point arithmetic may require testing and careful review.

If Simile chooses to represent numbers through conventional fixed point arithmetic, struggles with overflow and extremely large numbers are compounded by the innate non-determinism within Simile. For example, calculating the sum of a set of very large numbers can be done in arbitrary order, according to the semantics of mathematics (and Simile). But in fixed-point arithmetic, the resulting value can very well differ based on the order of operations. In other words, commutative math operations are no longer commutative under fixed precision. At some point in the computation pipeline, Simile will need to decide the order in which to evaluate the set, which may cause hard-to-spot bugs and unexpected runtime failures.

Conversely, choosing arbitrary precision integers may bring Simile operators closer to mathematics at the cost of speed. The core features of Simile, while targeting formal methods users, attempt to efficiently execute set and relation operations. Adding checks for overflow on every mathematical operation may prove detrimental to performance.

The implementation decision essentially is one of context vs. safety. To benchmark Simile with these types, we gather a list of examples and judge whether the performance hit is worth the reliability benefits. Specific questions to examine: How often does overflow occur in these examples? Do numbers approach the upper limits of fixed precision often enough to warrant accommodating infrastructure? What is the performance of fixed vs. arbitrary precision calculations.

Floating Point Numbers In a similar vein to integers, floats are an imperfect translation of real numbers into efficiently calculable representations. Since floats (or even reals) are not used as often in specifications as integers, we leave the implementation as the conventional 64-bit representation. However, we note that non-determinism on these operations will prove especially tricky, since the result of commutative complements may differ without explicit over/underflow.

Strings Strings are syntactic sugar over Sequences of characters, represented internally as a sequence of (fixed precision) integers. Eventually, Simile aims to support built-in string operations as seen in many common programming languages.

4.1.2 Sets

Foundational Simile type: an unordered, unique collection of objects.

Sets may be used in two ways:

1. Enumerations by way of $\{ x, y, z \}$ (read: The set of elements x, y, and z)
2. Comprehensions of the form $\{ x \in S \cdot p(x) \mid f(x) \}$ (read: The set of function f applied to all elements in S satisfying predicate $p(x)$). A small effort is made to accommodate a shorthand form, wherein $\{ x \in S \cdot true \mid x \}$ may be written as $\{ x \in S \mid x \}$.

Sets, as mathematical objects with only two major restrictions, lend themselves well to a variety of computational implementations. Arrays, ordered arrays, hash maps, bit sets, roaring bit sets, path maps, and tries can all be used to satisfy the uniqueness property and hide an internally ordered representation from users.

No matter the underlying implementation, the optimizer may make use of syntactically-analyzed set sizes to direct the choice of lowered iterator. Further, recording the cardinality of a set at runtime may prove beneficial to the performance of common cardinality-based specification expressions at a very small memory cost. Other properties based on element type, like max/min for numeric types, limits for numeric ranges, maximum sizes, may be useful.

4.1.3 Relations

Relations are sets that make use of paired/maplet elements (i.e., they define a relationship between two sets). Like sets, implementations may vary greatly, with the potential to increase efficiency of relation-specific operations.

Like sets, relations may benefit from compile-time computed properties: size of the domain, size of the range, totality, many-to-oneness, $O(1)$ access to the domain and range, and access to associated domain/range properties.

Relational Subtypes The notion of a relation is a broad classification of a set of pairs, where the domain, range, and relationship are not bound by any conditions. However, when generating code based on purely mathematical relations, it can be advantageous to use notions of totality, injectivity, surjectivity, etc. In particular, these named relational subtypes can be described with four properties: total on the domain, total on the range, one-to-many, and many-to-one.

The below table converts conventional notation into these four subtypes:

Relational Subtype	Properties			
	T	TR	1-	1-R
Relation				
				✓
Partial Function			✓	
Partial Injection			✓	✓
Surjective Relation		✓		
		✓		✓
Partial Surjection		✓	✓	
		✓	✓	✓
Total Relation	✓			
	✓			✓
Total Function	✓		✓	
Total Injection	✓		✓	✓
Total Surjective Relation	✓	✓		
	✓	✓		✓
Total Surjection	✓	✓	✓	
Bijection	✓	✓	✓	✓

Table 7: Conventional mathematical notation of relational subtypes decomposed into four properties: domain totality (T), range totality (TR), domain one-to-manyness (1-), and range one-to-manyness (1-R)

Subtype Properties Properties that can affect code generation may involve:

- New properties after applying operations on sets/relations
- Size of result
- Validity of operations
- Deterministic nature of operations
- Super/subset properties
- Relational identities

Below is a list of the impact of these properties.
Domain Totality:

- $R[S] \neq \varnothing$
- $R(S)$ is always valid
- R^{-1} is TR
- $|R| \geq dom(R)$

Range Totality:

- R^{-1} is T
- $|R| \geq ran(R)$

Domain One-to-manyness:

- $R(\{\, e \,\})$ is deterministic
- $|R[\{\, e \,\}]| = 1$
- $|R[S]| \leq |S|$
- $|R| \leq dom(R)$

Range One-to-manyness:

- $|R| \leq ran(R)$

4.1.4 Bags

Bags (or Multisets) are a refinement of relations wherein, for a bag with element type T, the bag is a relational function from $T \to \mathbb{Z}^+$, denoting the count of items in a collection. Once the count of an item reaches 0, the item is removed from the bag.

Bags must be many-to-one and carry specialized behaviours for conventional relation operations. For example, relational image ($R[B]$) with a bag argument will now return a bag, where the number of occurrences of the resulting item replaces what would otherwise be a regular set of those items.

Additional fields to record the size of the bag (sum of the range) may prove useful.

4.1.5 Sequences

Sequences are yet another refinement of relations where a sequence with element type T represents a relational function from $\mathbb{N} \to T$ that is total on some defined bounds.

The dense nature of sequence domains, along with knowledge of sequence sizes, may open up further data type refinements.

4.2 Data Type Operations

4.2.1 Primitives

4.2.2 Sets

4.2.3 Relations

4.2.4 Bags

4.2.5 Sequences

5 AST Lowering and Optimization Rules

Below is a list of rewrite rules for key abstract data types, syntactic sugar, and some builtin functions. Phases are intended to be executed in order; the post-condition of one phase serves as the pre-condition for the next. Standard functions (like map, flatten, length, etc.) are written as a library in Simile rather than baked-in optimizations.

5.1 Syntactic Sugar for Bags

$$R[B] \rightsquigarrow R^{-1} \circ B \qquad \text{(Bag Image)}$$

$$S \cup T \rightsquigarrow [\![\, x \in dom(S) \cup dom(T) \cdot n = max(S[x] \cup T[x]) \mid x \mapsto n \,]\!] \qquad \text{(Bag Predicates - Union)}$$

$$S \cap T \rightsquigarrow [\![\, x \in dom(S) \cdot x \in dom(T) \wedge n = min(S[x] \cup T[x]) \wedge n > 0 \mid x \mapsto n \,]\!] \qquad \text{(Bag Predicates - Intersection)}$$

$$S + T \rightsquigarrow [\![\, x \in dom(S) \cup dom(T) \cdot n = sum(S[x] \cup T[x]) \mid x \mapsto n \,]\!] \qquad \text{(Bag Predicates - Sum)}$$

$$S - T \rightsquigarrow [\![\, x \in dom(S) \cdot n = S(x) - (T \cup [\![\, x \mapsto 0 \,]\!])(x) \wedge n > 0 \mid x \mapsto n \,]\!] \qquad \text{(Bag Predicates - Difference)}$$

5.2 Syntactic Sugar for Sequences

$$L +\!\!+ M \rightsquigarrow L \cup \{\, x \mapsto y \in M \mid x + max(dom(L)) \mapsto y \,\} \qquad \text{(Concatenation)}$$

5.3 Syntactic Sugar for Relations

$$f(x) := E \rightsquigarrow f := f \oplus \{\, x \mapsto E \,\} \qquad \text{(Functional Override)}$$

$$R \oplus Q \rightsquigarrow Q \cup (dom(Q) \lhd R) \qquad \text{(Override)}$$

$$S \lhd R \rightsquigarrow \{\, x \mapsto y \in R \cdot x \in S \mid x \mapsto y \,\} \qquad \text{(Domain Restriction)}$$

$$S \lhd\!\!\!- R \rightsquigarrow \{\, x \mapsto y \in R \cdot x \notin S \mid x \mapsto y \,\} \qquad \text{(Domain Subtraction)}$$

$$R \rhd S \rightsquigarrow \{\, x \mapsto y \in R \cdot y \in S \mid x \mapsto y \,\} \qquad \text{(Range Restriction)}$$

$$R \rhd\!\!\!- S \rightsquigarrow \{\, x \mapsto y \in R \cdot y \notin S \mid x \mapsto y \,\} \qquad \text{(Range Subtraction)}$$

5.4 Comprehension Construction

Intuition All set-like variables and literals are decomposed into set comprehensions.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.

$$R(x) \rightsquigarrow \mathit{choice}(R[x]) \quad \text{(Functional Image } {}^{a}\text{)}$$
$$R[S] \rightsquigarrow \{\, x \mapsto y \in R \cdot x \in S \mid y \,\} \quad \text{(Image)}$$
$$x \mapsto y \in S \times T \rightsquigarrow x \in S \wedge y \in T \quad \text{(Product)}$$
$$x \mapsto y \in R^{-1} \rightsquigarrow y \mapsto x \in R \quad \text{(Inverse)}$$
$$x \mapsto y \in (Q \circ R) \rightsquigarrow x \mapsto z \in Q \wedge z' \mapsto y \in R \wedge z = z' \quad \text{(Composition)}$$

[a] May fail if R is not total. *choice* is nondeterministic.

$$S \cup T \rightsquigarrow \{\, (x \in S \mid x) \text{`} (x \in T \cdot x \notin S \mid x) \,\} \quad \text{(Predicate Operations - Union)}$$
$$S \cap T \rightsquigarrow \{\, x \in S \mid x \in T \,\} \quad \text{(Predicate Operations - Intersection)}$$
$$S \setminus T \rightsquigarrow \{\, x \in S \mid x \notin T \,\} \quad \text{(Predicate Operations - Difference)}$$
$$x \in \oplus(E \mid P) \rightsquigarrow P \wedge x = E \quad \text{(Membership Collapse } {}^{a}\text{)}$$
$$e \in S \rightsquigarrow \exists x \in S \mid e = x \quad \text{(Set Membership)}$$

[a] Rule only matches inside the predicate of a quantifier. Explicitly enumerating all matches for all quantuantification types and predicate cases (ANDs, ORs, etc.) would require too much boilerplate. x must be bound by the encasing quantifier.
The $\oplus$ operator represents any quantifier that returns a set-like type (ex. generalized union/intersection, set comprehension, relation comprehension).

5.5 Disjunctive Normal Form

Intuition All quantifier predicates are expanded to DNF (i.e. $\wedge$-operations nested within top-level $\vee$-operations).

Notes All matches of this phase occur only inside quantifier predicates.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Quantifier predicates are in disjunctive normal form - top level or-clauses with inner and-clauses.
- If no $\vee$ operators exist within a quantifier predicate, the predicate must only contain $\wedge$ operators.

$$x_1 \wedge ... \wedge (x_i \wedge x_{i+1}) \wedge ... \rightsquigarrow x_1 \wedge ... \wedge x_i \wedge x_{i+1} \wedge ... \quad \text{(Flatten Nested } \wedge\text{)}$$
$$x_1 \vee ... \vee (x_i \vee x_{i+1}) \vee ... \rightsquigarrow x_1 \vee ... \vee x_i \vee x_{i+1} \vee ... \quad \text{(Flatten Nested } \vee\text{)}$$
$$\neg\neg x \rightsquigarrow x \quad \text{(Double Negation)}$$
$$\neg(x \vee y) \rightsquigarrow \neg x \wedge \neg y \quad \text{(Distribute De Morgan - Or)}$$
$$\neg(x \wedge y) \rightsquigarrow \neg x \vee \neg y \quad \text{(Distribute De Morgan - And)}$$
$$x \wedge (y \vee z) \rightsquigarrow (x \wedge y) \vee (x \wedge z) \quad \text{(Distribute } \wedge \text{ over } \vee\text{)}$$

5.6 Or-wrapping

Intuition Restructuring quantifier predicates with top-level ORs for later rules.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Quantifier predicates are in disjunctive normal form - top level or-clauses with inner and-clauses.

$$\{ E \mid \bigwedge P_i \} \rightsquigarrow \{ E \mid \bigvee \bigwedge P_i \} \qquad \text{(Or-wrapping [a])}$$

[a] To simplify the matching process later on, we wrap every top-level AND statement (which is guaranteed to be a ListOp by the dataclass field type definition) with an OR.

5.7 Generator Selection

Intuition All nested ANDs are placed in a tree structure to better suit loop lowering.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Each top-level or-clause within quantification predicates must have one selected 'generator' predicate (GeneratorSelectionPredicate - GSP) of the form $x \in S$ that loops over its bound dummy variable x.
- All conjunctive clauses are wrapped in either a GSP or CombGSP structure.
- Quantifier predicates are in disjunctive normal form (with GSP replacing AND structures) - top level or-clauses with inner and-clauses.

Notes All matches of this phase occur only inside quantifier predicates. GSP is a tuple of form (generator chain, predicates) that can be flattened to a conjunctive clause. A CombGSP is a triple of form (shared generator, disjunctive children predicates/generators, predicates) that can likewise be flattened into a conjunctive clause. We keep the structure of these tuples distinct to ease the rewriting process.

Brainstorming selection heuristics Conditions to consider:

- Prefer composition chains in case of nested loops (ex. $x \mapsto y \in R \wedge y \mapsto z \in Q$, but what about renaming? - $x \mapsto y \in R \wedge y' \mapsto z \in Q \wedge y = y'$). Perhaps this could just be a preference for relations over sets if we see a nested quantifier.
- Choose most common generator among a list of or-clauses (allows for greater simplification later on)
- Choose smallest generator (by set size). This information may not always be accessible (ex. if a set is created by reading values from standard input). Functions may need to choose this on a case-by-case basis (ie. one function call could have two args, with a small set on the left arg and a large set on the right arg. But what if the sizes are reversed later in the code? We've already statically lowered it).
- When should constant folding happen? Equality substitution necessary for this?

- This may need to work with nesting considerations.
- What if we try moving these optimizations to *loop* structures far later in the pipeline?

$$\bigwedge P_i \rightsquigarrow GSP(\bigwedge P_{g_i}, \bigwedge_{P_i \notin P_g} P_i) \qquad \text{(GSP Wrapping [a])}$$

$$GSP(x \wedge xs, P) \vee GSP(x \wedge ys, Q) \rightsquigarrow CombGSP(x, GSP(xs, P) \vee GSP(ys, Q)) \qquad \text{(Nested Generator Selection [b])}$$

[a] GSP is a tuple-like structure, where all entries can be flattened into an AND. The LH term must occur inside a quantifier's predicate - one generator per top-level or-clause. Chained generators may be necessary to allow for nested loop operations (ie. non-top-level clauses may require generators). P_g is a list of chained generators, specific set membership clauses (of form $x \in S$) distinguished from the rest of $\bigwedge P_i$. Currently, the selection of P_g is arbitrary (and thus the rewrite system is not confluent), but heuristics may be added later to choose better generators.

[b] $free$ is a function that returns true when variables in the predicate are defined (either bound by the current generator or bound prior to the current statement)

5.8 GSP to Loop

Intuition Start lowering expressions into imperative-like loops.

Post-condition

- All quantifiers are transformed into *loop* structures.
- *loop* predicates are of the form $x \in S \wedge \bigwedge P_i$.
- All *loop* predicates have an assigned generator of the form $x \in S$.

$$\oplus E \mid P \rightsquigarrow \begin{array}{l} a := identity(\oplus) \\ \textbf{loop } P: \\ \quad a := accumulate(a, E) \end{array} \qquad \text{(Quantifier Generation [a])}$$

[a] $\oplus$ works for any quantifier (but not $\forall$ and $\exists$). The identity and accumulate functions are determined by the realized $\oplus$. For example, if $\oplus = \sum$, the identity is 0 and accumulate is addition.

$$\begin{array}{l} \textbf{loop } \bigvee P_i: \\ \quad \text{body} \end{array} \rightsquigarrow \begin{array}{l} \textbf{loop } P_0 \\ \quad body \\ \textbf{loop } P_1 \wedge \neg P_0 \\ \quad body \\ \textbf{loop } P_2 \wedge \neg \bigvee_{i<2} P_i \\ \quad body \end{array} \qquad \text{(Top-level Or-Loop)}$$

$$\begin{array}{l} \textbf{loop } GSP(g \wedge gs, P) \\ \quad \text{body} \end{array} \rightsquigarrow \begin{array}{l} \textbf{loop } SingleGSP(g, free(P)) \\ \quad \textbf{loop } GSP(gs, bound(P)) \\ \quad\quad \text{body} \end{array} \qquad \text{(Chained GSP Loop [a])}$$

[a] $free$ considers defined variables at this point in time.

loop $GSP([], P)$
  body

$\rightsquigarrow$

if P **then**
  body

(Empty GSP Loop)

loop $CombGSP(g, gs, P)$
  body

$\rightsquigarrow$

loop $SingleGSP(g, P)$
  loop gs_0
    body
  loop gs_1
    body
  ...

(Combined GSP Loop)

5.9 Relational Subtyping Loop Simplification

Intuition Simplify unnecessary iteration structures into direct accesses.

Post-condition

- All quantifiers are transformed into *loop* structures.
- *loop* predicates are of the form $x \in S \wedge \bigwedge P_i$.
- All *loop* predicates have an assigned generator of the form $x \in S$.

No change from previous, but the number of loop structures should not increase.

$x \in dom(R) \rightsquigarrow true$ (Total membership elimination [a])

$x \in ran(R) \rightsquigarrow true$ (Surjective membership elimination [b])

[a] R is total and x satisfies the type of R's domain. Currently unused since it may eliminate some generators - move higher in the list?
[b] R is surjective and x satisfies the type of R's range. Currently unused since it may eliminate some generators - move higher in the list? Or

loop $SingleGSP(x \mapsto y \in R, x = a \wedge P)$
  body

$\rightsquigarrow$

if $a \in dom(R)$ **then**
  loop $SingleGSP(y \in R[a], P[x := a])$
    body

(Concrete Domain Image)

loop $SingleGSP(x \mapsto y \in R, y = a \wedge P)$
  body

$\rightsquigarrow$

if $a \in ran(R)$ **then**
  loop $SingleGSP(x \in R^{-1}[a], P[y := a])$
    body

(Concrete Range Image [a])

[a] Requires efficient lookups for ran, inverse, and image

loop $SingleGSP(y \in R[x], P)$
  body

where $free(x)$ and R is many-to-one

$\rightsquigarrow$

if $P[y := R(a)]$ **then**
  $body[y := R(a)]$

(Single Element Loop)

loop $SingleGSP(x \in \{\, y \,\}, P)$
  body

$\rightsquigarrow$

if $P[x := y]$ **then**
  $body[x := y]$

(Singleton Membership Elimination)

5.10 Loop Code Generation

Intuition Eliminate all intermediate loop structures.

Post-condition

- AST is in imperative code style (for loops, if statements, etc).
- All *loop* and quantification constructs have been eliminated.
- Some variables may not be defined.

loop $SingleGSP(P_g, \bigwedge P_i)$
  body

$\rightsquigarrow$

if $\bigwedge_{free(P_i)} P_i$ **then**
  for P_g **do**
    if $\bigwedge_{bound(P_i)} P_i$ **then**
      body

(Conjunct Conditional [a])

[a] Function $free$ returns clauses in P that contain only free + defined variables. *bound* returns the clauses that contain the bound variable x or undefined variables. P_g is the selected generator.

5.11 Replace and Simplify

Intuition Eliminate all undefined variables (with implicit $\exists$ quantifiers).

Post-condition

- AST is in imperative code style (for loops, if statements, etc).
- All variables are defined.

if $Identifier(x) = E \wedge P$ **then**
  body

$\rightsquigarrow$

if $P[x := E]$ **then**
  body[x := E]

(Equality Elimination [a])

[a] x is an undefined, unbound variable in the current scope. E is an expression that does not contain x. Simplify resulting booleans.

$$P \wedge true \rightsquigarrow P \qquad \text{(Simplify And-true)}$$
$$P \wedge false \rightsquigarrow false \qquad \text{(Simplify And-false)}$$
$$P \vee true \rightsquigarrow true \qquad \text{(Simplify Or-true)}$$
$$P \vee false \rightsquigarrow P \qquad \text{(Simplify Or-false)}$$
$$Statement(Statement(x)) \rightsquigarrow Statement(x) \qquad \text{(Flatten Nested Statements)}$$